



RV College of Engineering[®]

Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

IoT based Smart Irrigation in Arecanut Farms

MAJOR PROJECT REPORT

MCA491P

submitted by

HARI KETAN T 1RV23MC039

under the guidance of

Dr. Renuka Prasad B

Associate Professor

Department of MCA

in partial fulfillment for the award of degree of

Master of Computer Applications

Department of Master of Computer Applications

2024-2025





RV College of Engineering®

Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

www.rvce.edu.in
Tel: +91-80-68188100
+91-80-68188111
+91-80-68188112

DEPARTMENT OF MASTER OF COMPUTER APPLICATIONS

IOT BASED SMART IRRIGATION IN ARECANUT FARMS

MAJOR PROJECT REPORT

MCA491P

submitted by

HARI KETAN T 1RV23MC039

under the guidance of

Dr. Renuka Prasad B

Associate Professor

Department of MCA

in partial fulfilment for the award of degree of

MASTER OF COMPUTER APPLICATIONS

2024-2025



RV College of Engineering®

Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

CERTIFICATE

Certified that the Major Project titled **IoT based Smart Irrigation in Arecanut Farms** is carried out by **Hari Ketan T (1RV23MC039)** a bonafide student of RV College of Engineering, Bengaluru, in partial fulfillment for the award of Degree of **Master of Computer Applications** of **Visvesvaraya Technological University, Belagavi** during the year **2024-2025**. It is certified that all corrections/suggestions indicated for the internal assessment have been incorporated in the report deposited in the department library. The project report has been approved as it satisfies the academic requirements in respect of project work prescribed by the institution for the said Degree.

B. Renuka Prasad 6/8/25

Internal Guide

Dr. Renuka Prasad B

Associate Professor

Department of MCA

RV College of Engineering

Jasmine K S

Head of the Department

Dr. Jasmine K S

Associate Professor & Director

Department of MCA,

RV College of Engineering

Subramanya

Principal

Dr. K N Subramanya

External Viva Examination

Name of Examiners

Signature with Date

1.

2.



CISCO-RVCE-CoE-IoT



PROJECT COMPLETION CERTIFICATE

This is to certify that **Hari Ketan T (1RV23MC039)**, of Department of MCA, RV College of Engineering, has successfully completed the project titled "IoT Based Smart Irrigation System in Arecanut Farms"

This project was undertaken as a part of the CISCO RVCE Lab and was funded by BSOL Solutions, during the period from April 2025 to August 2025.

B. Renuka Prasad
2/8/25
Co-ordinator

CISCO-RVCE-CoE-IoT

Dr. Renuka Prasad B
Associate Professor
Department of MCA
RV College of Engineering



RV College of Engineering®

Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

DECLARATION

I **Hari Ketan T**, the student of Fourth semester Department of MCA, RV College of Engineering*, Bengaluru-560059, bearing USN: **1RV23MC039** hereby declare that the project titled **IoT based Smart Irrigation in Arecanut Farms** has been carried out by me. It has been submitted in partial fulfilment of the program requirements for the award of Degree in **Master of Computer Applications** of **Visvesvaraya Technological University, Belagavi** during the year **2024-2025**.

Further, I declare that the content of the dissertation has not been submitted previously by anybody for the award of any Degree or Diploma to any other University.

I also declare that any **Intellectual property rights generated out of this project carried out at RVCE** will be the property of **RV College of Engineering*, Bengaluru** and I will be among the authors of the same.

Place: Bengaluru

Date of Submission: 06/08/2025


Signature of the Student

Hari Ketan T

1RV23MC039

Department of Master of Computer Applications

RV College of Engineering*

Bengaluru - 560059

ACKNOWLEDGEMENT

The successful completion of the major project titled IoT-Based Smart Irrigation System for Arecanut Farms has been made possible through the support and guidance of many individuals and institutions.

Special thanks are offered to the esteemed Principal, **Dr. K.N. Subramanya**, RV College of Engineering for his visionary leadership and continued support of student-led innovation.

Deep gratitude is also expressed to **Dr. Jasmine K. S**, Associate Professor and Director, Department of Master of Computer Applications for her guidance throughout the project duration. Their belief in the capabilities of the students has been a driving force behind the successful execution of this initiative.

Heartfelt appreciation is extended to the project guide, **Dr. Renuka Prasad B**, Associate Professor, whose technical expertise, patience, and thoughtful mentorship have played a pivotal role in shaping the project's direction. Her detailed feedback and strategic inputs have greatly contributed to overcoming challenges and enhancing the system's design and functionality.

This project has provided a valuable platform for learning and practical application of IoT technologies in agriculture. It is hoped that the Smart Irrigation System will serve as a meaningful contribution toward promoting sustainable farming practices and efficient water resource management in arecanut cultivation.

Hari Ketan T (1RV23MC039)

Department of MCA

RV College of Engineering®

Bengaluru - 560059

ABSTRACT

This project tackles a major issue in arecanut farming the heavy dependence on manual irrigation, which often leads to overwatering, underwatering, and waste of labor and water resources. Arecanut plants require regular and precise watering, and even small inefficiencies can affect crop health and yield. To address this, the system integrates Internet of Things (IoT) technologies to automate and optimize irrigation.

The setup begins at the field level with capacitive soil moisture sensors that measure how dry or wet the soil is, and a YF-S201 flow sensor that monitors how much water is actually delivered. These are connected to an ESP32 microcontroller, which collects the data and transmits it wirelessly via the MQTT protocol to a Raspberry Pi the system's central brain. From there, Python scripts on the Raspberry Pi interpret the sensor readings and compare them against a Water Requirement Table that you've created, which is customized based on plant age, soil type, and seasonal needs. When the soil moisture drops below the desired level, the Raspberry Pi sends a command to open a motorized valve using a Wi-Fi relay hub, initiating irrigation only when it's truly needed. This helps maintain optimal soil conditions while reducing unnecessary water use.

All data including moisture levels, irrigation events, and system status is stored in a PostgreSQL database and displayed in real-time on a Next.js-based web dashboard. This interface allows farmers to monitor their fields from anywhere, manually control irrigation if needed, and view historical trends to make better farming decisions. By combining affordable hardware with intelligent software, the system enables precision irrigation without requiring advanced technical skills from the farmer. It not only saves up to 30% of water, but also reduces manual labor and contributes to sustainable agriculture, making it a promising solution for small and medium-sized arecanut farms.

Table of Contents

CONTENTS	Page No
College Certificate	i
Company/ CoE Certificate	ii
Declaration by student	iii
Acknowledgement	iv
Abstract	v
Table of Contents	vi
List of Tables	viii
List of Figures	ix
 Chapter 1: Introduction	 01
1.1 Project Description	01
1.2 Dissertation Organization	03
 Chapter 2: Literature Review	 05
2.1 Literature Survey	05
2.2 Existing and Proposed System	08
2.3 Tools and Technologies used	09
2.4 Hardware and Software Requirements	12
 Chapter 3: Software Requirement Specifications	 14
3.1 Introduction	14
3.2 General Description	15
3.3 Functional Requirement	19
3.4 Non-Functional Requirements	21
3.5 Design Constraints	22
 Chapter 4: System Design	 25
4.1 System Perspective	25
4.2 Context Diagram	33

Chapter 5: Detailed Design	35
5.1 System Design	35
Class Diagram	36
Use case Diagram	37
Activity Diagram	38
Sequence Diagram	39
Data Flow Diagrams	40
5.2 Detailed design	43
 Chapter 6 Implementation	 45
6.1 Code Snippets	45
6.2 Implementation	47
 Chapter 7: Software Testing	 49
7.1 Test cases	49
7.2 Testing and Validations	51
 Chapter 8: Conclusion	 53
 Chapter 9: Future Enhancements	 54
 Bibliography	 55

List of Tables

Table No	Title	Page No
2.1	Hardware Requirement of Smart Irrigaton in Arecanut Farms	12
2.2	Software Requirement of Smart Irrigation inArecanut Farms	13
7.1	Test Case On Soil Moisture Monitoring	49
7.2	Test Case on Irrigation Control (Auto/Manual)	50
7.3	Test Case on Water Flow Monitoring	50
7.4	User Login for Smart Irrigation	50

List of Figures

Figure No	Title	Page No
2.1	Mind Map of the Literature Survey	07
3.1	Circuit Connection of the Hardware Components	24
4.1	Representation of Block Diagram of IoT based Smart Irrigation	26
4.2	Context Diagram of Smart Irrigation	33
5.1	Representation of Class Diagram of IoT Based Smart Irrigation	36
5.2	Representation of Use-Case Diagram for IoT based Smart Irrigation	37
5.3	Representation of Activity Diagram of Smart Irrigation	38
5.4	Sequence Diagram for Smart Irrigation	39
5.5	Data Flow Diagram of Smart Irrigation – Level 0	40
5.6	Data Flow Diagram of Smart Irrigation – Level 1	41
5.7	Data Flow Diagram of Smart Irrigation – Level 2	42
5.8	Data Flow Diagram of Smart Irrigation – Level 2	43
6.1	Dashboard of the Smart Irrigation Application	47
6.2	Detailed Section of the Smart Irrigation Application	47
6.3	Farm Map For Smart Irrigation Application	48
6.4	Analytics Page For Smart Irrigation	48
7.1	Validation of User Credentials in Login page	51
7.2	Validations of User Registration in Register page	51
7.3	MQTTX Messages for Validating Irrigation Command	52

Chapter 1

Introduction

1.1 Project Description

The "IoT-Based Smart Irrigation System for Arecanut Farms" is a precision agriculture project conceived to address the acute challenges of water management in the cultivation of arecanut palms (*Areca catechu*). In many parts of Asia, and particularly in India, arecanut cultivation is a vital horticultural practice that forms the economic backbone for a multitude of small and medium-scale farmers. The project specifically addresses the need for intelligent systems in niche agricultural sectors, where technology can be adapted for specialized crops beyond common staples, sometimes even for tasks like crop disease prediction [1]. The profitability of this crop is intrinsically linked to a consistent and adequate supply of water, as its shallow root system makes it highly susceptible to both water stress and over-watering. The application of smart techniques, including data mining, is essential for the resource-efficient and sustainable production of such crops [2]. The future of sustainable agriculture lies in moving away from traditional, inefficient methods towards these kinds of intelligent, resource-managing systems [3].

The prevailing irrigation techniques in arecanut farming, such as manual flood irrigation or rudimentary sprinkler systems, are often inefficient and based on generalized schedules or farmer intuition. This approach leads to a cascade of negative consequences. Over-irrigation not only wastes a scarce natural resource but also creates waterlogged soil conditions that can lead to root rot and fungal diseases while causing essential nutrients to leach away from the root zone [4]. Conversely, under-watering induces significant moisture stress, which retards growth, reduces nut yield, and can severely impact the farm's financial viability [5]. Furthermore, these manual methods are exceptionally labor-intensive, demanding the farmer's physical presence to operate valves and pumps, which represents a traditional practice ripe for modernization through IoT [6].

This project introduces an automated, data-driven solution that leverages the Internet of Things to inject precision and efficiency into the irrigation process. The fundamental objective is to transition from a rigid, schedule-based paradigm to an intelligent, needs-based system that responds directly to the farm's real-time conditions. By deploying a network of sensors within the farm environment, the system can continuously monitor key parameters, primarily soil moisture, to make informed irrigation decisions [7]. It is designed to automatically activate the water supply only when moisture levels fall below a scientifically determined threshold and to deliver a precise, measured volume of

water, thereby preventing both waste and deficit [8]. This is a common goal in modern smart irrigation systems, often combined with other features like crop disease detection to provide a more holistic management tool [9]. Many contemporary IoT frameworks for agriculture aim to incorporate some level of machine learning, even if simple, to enhance this decision-making process [10]. The overarching goal is to create a system that automates the entire monitoring and irrigation cycle, providing farmers with a tool that works efficiently in the background [11].

The solution is architected as a complete, end-to-end system. A network of capacitive soil moisture sensors and water flow sensors gathers ground-truth data. This data is transmitted wirelessly by ESP32 microcontrollers using the lightweight MQTT protocol to a central Raspberry Pi gateway, which functions as an edge computing node. This gateway processes the information, applies a set of predefined rules based on crop age, soil type, and time of year, and makes an intelligent decision on whether to irrigate. Such platforms can be viewed as comprehensive water management solutions for precision agriculture [12]. For the farmer, the entire system is accessible through a user-friendly web dashboard that provides a real-time overview of the farm's conditions, historical data logs, and the ability to switch between automated control and manual override. This approach can be expanded with additional sensors to create a multi-purpose agricultural management system, capable of detecting intrusions or other events [13].

1.1.1 The Internet of Things (IoT) in Agriculture

The IoT paradigm, when applied to agriculture, shifts the management approach from treating a field as a uniform block to observing and responding to granular, intra-field variations. The framework for this project is structured in distinct layers. The Perception Layer is the physical layer, comprising the sensors that convert environmental phenomena into electrical signals. The Network Layer handles the transmission of this data from the sensors to a central unit. This project uses the MQTT (Message Queuing Telemetry Transport) protocol, which is exceptionally well-suited for IoT applications due to its lightweight nature and efficiency in low-bandwidth environments [14]. The Processing Layer is responsible for making intelligent decisions. Here, the Raspberry Pi serves as an edge computing device, processing data locally to reduce latency and ensure the system can function even without an internet connection, a concept central to modern edge-cloud hybrid architectures [15]. This layer also includes the Node.js backend and PostgreSQL database, which manage data storage and API services, a common debate in IoT being the choice between SQL and NoSQL databases for such tasks [16]. Finally, the Application Layer is the user-facing Next.js web

dashboard, which translates complex data into actionable insights and provides system controls, sometimes combined in other projects with image processing for enhanced crop monitoring [17].

1.1.2 Soil-Moisture-Based Irrigation Control

The system's intelligence is centered on its use of real-time soil moisture data rather than a fixed calendar schedule, a method proven to be far more efficient as it directly addresses the plant's actual water requirements [18]. The project uses capacitive soil moisture sensors, which measure the soil's dielectric permittivity to determine its water content. This approach is superior to older resistive sensors, which are prone to corrosion. The control logic itself is a straightforward but effective form of threshold-based control. A specific soil moisture level is defined as the threshold, but this value is dynamic and fetched from a `Water_Requirement` table. This table allows the threshold to be adjusted based on critical variables like plant age, soil type, and the current season, ensuring the irrigation strategy is scientifically grounded and adaptive [19].

1.2 Dissertation Organization

This dissertation is structured into nine distinct chapters to provide a clear and logical progression of the project from its conception through to its final evaluation and potential future.

- Chapter 1 introduces the project, establishing the context, defining the core problem in arecanut farming, presenting the proposed IoT solution, and outlining the organization of this report.
- Chapter 2 provides a comprehensive review of the relevant literature, surveying existing smart irrigation systems, comparing different technological approaches, and justifying the tools and technologies chosen for this project.
- Chapter 3 details the Software Requirement Specifications (SRS), formally defining the system's functional and non-functional requirements, interfaces, and operational constraints.
- Chapter 4 describes the high-level system architecture, illustrating the overall structure and flow of data and control through block diagrams and context diagrams.
- Chapter 5 presents the detailed design of the system using an object-oriented methodology, including UML diagrams (Class, Use Case, Activity, Sequence), Data Flow Diagrams, and the conceptual database schema.

- Chapter 6 focuses on the implementation phase, providing key code snippets and annotated screenshots to demonstrate how the design was translated into a functional prototype.
- Chapter 7 outlines the software testing process, detailing the unit, integration, and system testing methodologies adopted, along with the test cases used to validate the system's functionality.
- Chapter 8 presents the conclusion, summarizing the project's achievements, discussing the results, and evaluating how the system met its initial objectives.
- Chapter 9 explores potential Future Enhancements, suggesting avenues for expanding the system's capabilities and intelligence.

Chapter 2

Literature Review

2.1 Literature Survey

A review of the current literature confirms that the application of Internet of Things (IoT) in agriculture is a well-established and rapidly advancing field. Foundational surveys provide a clear overview of the role of IoT in implementing smart farming, outlining the key technologies and layered architectures that form the basis of modern agricultural systems [20]. These surveys emphasize the shift from traditional farming methods to data-driven precision agriculture. More specific reviews focus on the recent trends within smart irrigation, offering a detailed comparison of different sensor types (e.g., tensiometers, capacitive, resistive), communication protocols (e.g., LoRaWAN, Zigbee, Wi-Fi), and system architectures that are currently being explored [21]. A recurring theme across the literature is the powerful synergy between Artificial Intelligence (AI) and IoT in developing next-generation solutions for sustainable farming [22]. This synergy allows systems to move from simple automation to predictive and adaptive control. The primary outcome of these systems is consistently reported as more efficient water management, which is a critical goal for global agriculture given increasing water scarcity [23].

The broader scope of these technologies is often reviewed, highlighting how a well-designed IoT platform can serve as a foundation for a wide range of applications beyond just irrigation, such as pest control, crop monitoring, and automated harvesting [24]. Many recent implementations aim to leverage cloud computing for data storage and analysis, allowing for powerful processing and remote access capabilities that enable farmers to monitor their operations from anywhere [25]. The architectural choices often involve a hybrid model, combining local edge processing with cloud services to balance reliability with advanced features like deep learning [26]. The choice of database technology is also a critical consideration, with significant research comparing the performance of traditional SQL databases against NoSQL alternatives for handling the high-velocity, time-series data generated by IoT devices [27]. In many cases, these systems are enhanced with machine learning and image processing to provide a more holistic crop monitoring solution, allowing for the early detection of disease or nutrient deficiencies [28].

While general frameworks are well-documented, the most valuable insights come from case studies focused on specific crops, which demonstrate the practical application and benefits of these technologies. Soil moisture-based automation has been proven to be highly effective across a

variety of contexts, serving as the foundational principle for most modern smart irrigation systems [29]. A practical implementation of an IoT-based system demonstrates the core principles of sensing and actuation in a real-world scenario, often serving as a blueprint for more complex projects [30]. These general models are then adapted for specific needs. A foundational survey of IoT's role in agriculture confirms that customized solutions yield the best results, as different crops have vastly different water requirements and growth cycles [31]. An overview of recent trends in sensors and systems further reinforces the need for tailored hardware and software configurations to match the specific agricultural environment [32]. The goal of such tailored systems is to promote sustainable farming through the integration of AI and IoT, moving beyond simple automation to create truly intelligent farming operations [33]. This leads to demonstrable results in efficient water management across different system designs, as verified by numerous field trials and case studies [34]. A broad review of applications in the agriculture sector shows that irrigation is one of the most impactful use cases for IoT, offering immediate returns on investment through water and labor savings [35]. The implementation details often vary, with some systems relying entirely on cloud computing for their logic and data storage, which offers scalability but introduces dependencies on internet connectivity [36].

The success of an IoT-enabled system for bananas, which used soil moisture thresholds to improve yield, provides a strong model for other perennial crops, demonstrating that even simple threshold-based logic can be highly effective when correctly calibrated [37]. Similarly, a tailored system for mango orchards highlighted the importance of including plant-specific parameters like tree age and canopy size in the decision-making logic, as a plant's water needs change dramatically throughout its lifecycle [38]. An IoT system implemented in date palm groves demonstrated significant water savings, providing a quantitative benchmark for the potential impact of such technologies in arid and semi-arid regions [39]. These crop-specific successes underscore a critical point: while the technology is versatile, its effectiveness is maximized when it is precisely calibrated to the unique horticultural needs of the target crop. However, a notable gap exists in the literature regarding arecanut cultivation, with very few studies focused on the specific technological and agricultural challenges of this particular crop [40].

2.1.1 Summary of Literature Survey

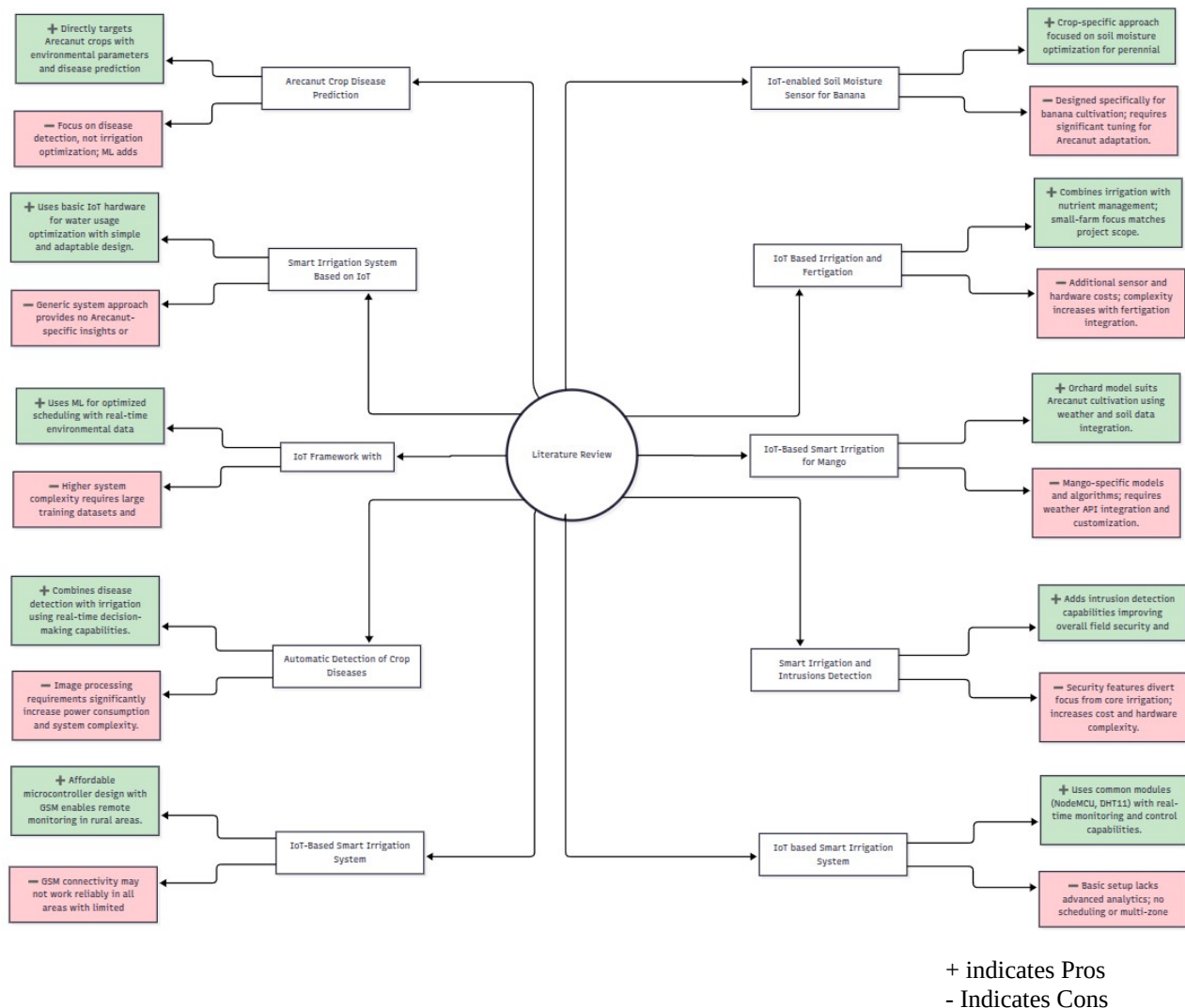


Figure 2.1 Mind Map of the Literature Survey

• Unresolved Issues and Emerging Opportunities

Despite the significant progress in smart irrigation, this literature survey reveals a critical unresolved issue: the lack of crop-specific research for arecanut. While numerous studies focus on staple crops or other fruits, arecanut, with its unique horticultural needs and economic importance in certain regions, remains largely unaddressed. This presents a clear opportunity to develop a tailored solution. Another key challenge is the trade-off between the complexity of advanced systems and their practical usability for small-scale farmers. This opens an important opportunity for systems like the one proposed, which use simple yet effective rule-based logic, providing an accessible entry point into precision agriculture.

- **Conclusion of Literature Survey and Project Motivation**

The comprehensive literature survey confirms that IoT technology provides a robust framework for smart irrigation. The reviewed studies collectively demonstrate that sensor-based, automated systems lead to significant benefits. However, the survey also highlights a critical gap: the absence of an accessible, affordable, and scientifically calibrated smart irrigation solution designed specifically for arecanut farming. Foundational software engineering principles suggest that addressing such a specific user need is a key driver for a successful project [41]. The primary motivation for this project is, therefore, to bridge this gap. By leveraging established IoT principles, as detailed in various case studies [42], to build a system that is practical, affordable, and directly relevant to the needs of arecanut farmers, this project aims to translate the proven potential of IoT into a tangible and impactful tool for a specific, underserved farming community.

2.2 Existing and Proposed System

2.2.1 Problem Statement

The conventional irrigation methods used in arecanut farming are predominantly manual and inefficient. Farmers often rely on intuition or fixed schedules, leading to significant water wastage, compromised crop health from over- or under-watering, and a heavy dependency on manual labor. The core problem is the absence of an affordable, data-driven technology that is specifically designed for the horticultural needs of the arecanut palm. Modern smart farming platforms often prioritize either cloud-based complexity or are too generic, failing to meet the reliability and cost requirements of smallholder farms [43].

2.2.2 The Scope of the Project

To address this, we are building a functional prototype with a clear focus:

- **Single-Zone Management:** The system is designed to control the irrigation for a single, defined farm section.
- **Essential Monitoring:** It monitors soil moisture and measures the precise volume of water used.
- **Simple, Rule-Based Intelligence:** The system uses a straightforward set of rules based on plant age, soil type, and time of year, avoiding the complexity of AI.
- **Offline-First and Low-Cost:** The system runs on-site, ensuring reliability without internet and eliminating recurring cloud fees.

- **Intuitive Dashboard:** A simple web interface provides real-time monitoring and manual control.
- **Data Logging:** The system automatically logs approximately one month of data for analysis.

2.2.3 Methodology Adopted in the Proposed System

The project followed the Prototyping Model, an iterative methodology well-suited for projects involving hardware and software integration. This approach, which emphasizes building and refining an early working model based on testing and user feedback, is a standard practice in modern software development.

Technical Features of the Proposed System

- **Crop-Specific, Multi-Factor Rule-Based Logic:** The system's intelligence lies in a dynamic rule engine that considers plant age, soil type, and month to deliver a customized and horticulturally sound watering regimen.
- **Hybrid Edge-On-Premises Architecture:** The system leverages both edge computing and on-premises deployment, ensuring reliability and low operational costs.
- **Closed-Loop Control with Volumetric Verification:** The system uses a water flow sensor to deliver a precise volume of water, offering superior accuracy and enabling anomaly detection [44].
- **Modular, Full-Stack Polyglot Architecture:** The system is built with a modular design using a multi-language technology stack, which enhances maintainability and makes the system easier to extend with future functionalities.

2.3 Tools and Technologies used

2.3.1 Hardware Platforms

- **Raspberry Pi (Model 4B):**
 - **Role:** Served as the central gateway and edge computing node of the system. It hosts the MQTT broker, runs the main Python control logic scripts, and communicates with the backend server.
 - **Justification:** Chosen for its powerful processing capabilities in a small form factor, extensive GPIO pins for potential future hardware integration, robust Linux-based operating system (Raspberry Pi OS), and strong community support. Its ability to run a

full software stack (broker, scripts, server) makes it an ideal, low-cost hub for the on-premises architecture.

- **ESP32 Microcontroller:**

- Role: Acted as the distributed sensor node controller. Each ESP32 is responsible for collecting data from its connected soil moisture and water flow sensors and publishing this data to the MQTT broker over Wi-Fi.
- Justification: The ESP32 is an ideal choice due to its integrated Wi-Fi and Bluetooth capabilities, dual-core processor, ultra-low power consumption, and affordability. These features make it perfectly suited for remote, battery-powered (if needed) deployment in the field.

2.3.2 Software Technologies and Frameworks

- **Next.js (Frontend Framework):**

- Role: Used to build the responsive, interactive web-based dashboard for the farmer. The dashboard provides real-time data visualization, historical data charts, and manual control over the irrigation system.
- Justification: Next.js, a React framework, was selected for its performance benefits (e.g., server-side rendering), excellent developer experience, and its ability to create modern, dynamic user interfaces that can handle real-time data updates efficiently.

- **Node.js with Express.js (Backend Framework):**

- Role: Formed the core of the backend API. The server handles HTTP requests from the user interface, processes commands, interacts with the PostgreSQL database, and receives data from the Raspberry Pi gateway.
- Justification: Node.js's event-driven, non-blocking I/O model is highly efficient for handling concurrent connections from multiple IoT devices and user sessions. Express.js provides a minimalist and flexible framework for building the robust APIs required for the system.

- **PostgreSQL (Database Management System):**

- Role: The primary data repository for the system. It is used to store all time-series data from the sensors (soil moisture, water flow), system event logs (valve ON/OFF), and farm configuration details.
- Justification: PostgreSQL is a powerful, open-source object-relational database renowned for its reliability, data integrity, and robust feature set. Its strong support for

timestamp-based data and complex queries is essential for logging, analysis, and generating historical reports.

- **Python (for Control Logic):**
 - Role: The language used to implement the core rule-based scheduling and threshold-based control algorithms that run on the Raspberry Pi gateway.
 - Justification: Python was chosen for its clean syntax, rapid development cycle, and extensive libraries, which make it ideal for scripting the system's decision-making logic.

2.3.3 Communication Protocol

- **MQTT (Message Queuing Telemetry Transport):**
 - Role: The communication protocol used for data exchange between the ESP32 sensor nodes and the Raspberry Pi gateway. The system uses Mosquitto, a popular open-source message broker, to manage MQTT communications.
 - Justification: MQTT is the de facto standard for IoT messaging. Its lightweight, publish-subscribe model is designed for low-bandwidth, high-latency, and unreliable networks, ensuring efficient and reliable data transmission with minimal overhead on the resource-constrained ESP32 devices.

2.3.4 Development and Programming Tools

- **Arduino IDE / PlatformIO IDE:**
 - Role: The Integrated Development Environment used for programming, compiling, and uploading the firmware to the ESP32 microcontrollers.
 - Justification: These IDEs provide a complete toolchain and necessary libraries for interfacing with various sensors and managing Wi-Fi connectivity on the ESP32 platform, simplifying the firmware development process.
- **Visual Studio Code (VS Code):**
 - Role: The primary source-code editor used for developing the backend (Node.js), frontend (Next.js), and Python control scripts.
 - Justification: VS Code is a versatile and powerful editor with excellent support for multiple languages, integrated debugging tools, a built-in terminal, and seamless Git integration, making it ideal for managing a full-stack, polyglot project.

2.4 Hardware and Software Requirement

2.4.1 Hardware Requirement

Table 2.1 Hardware Requirement Table of Smart Irrigaton in Arecanut Farms

Sl. No.	Hardware	Purpose
1	Raspberry Pi 4 Model B	The core processing unit of the system. It functions as an edge computing node, hosting the MQTT broker, executing the primary Python control logic, and interfacing with the backend database and server.
2	ESP32 Development Board	A low-power microcontroller responsible for acquiring data from the attached sensors (soil moisture, water flow) and publishing it to the MQTT gateway via its built-in Wi-Fi module.
3	Capacitive Soil Moisture Sensor v2.0	The primary input sensor used to monitor real-time moisture levels in the soil's root zone. It provides the core data point for triggering irrigation cycles.
4	YF-S201 Water Flow Sensor	A sensor installed in the irrigation pipeline to measure the precise volume of water being delivered. This enables volumetric control and facilitates system anomaly detection (e.g., leaks).
5	12V DC Motorized Solenoid Valve	The primary electromechanical actuator that physically opens and closes to control the flow of water to the irrigation zone based on commands from the control system.
6	4 Channel Relay Module	An electromechanical switch that translates the low-voltage digital signals from the control system into the higher voltage required to operate the motorized valve, providing isolation and remote control capability.

2.4.2 Software Requirements

Table 2.2 Software Requirement for Smart Irrigation in Arecanut Farms

Sl. No	Software	Purpose
1	Raspberry Pi OS	The base operating system for the Raspberry Pi gateway. It provides a stable and robust Linux environment for running all necessary services and scripts.
2	Python 3.12	Used for scripting the core control logic on the Raspberry Pi. Implements the rule-based decision-making algorithm that governs irrigation cycles.
3	Next.js	Used for developing the user-facing web dashboard. It provides a responsive and interactive interface for real-time data visualization and manual system control.
4	Node.js with Express.js	Used for building the RESTful API backend. It manages data flow between the user interface, the database, and the Raspberry Pi gateway, handling user authentication and data processing.
5	PostgreSQL	The relational database used for all data persistence. It stores time-series sensor data, system event logs, farm configurations, and user credentials.
6	Mosquitto	A lightweight message broker that runs on the Raspberry Pi. It manages the publish-subscribe communication between the ESP32 sensor nodes and the central gateway, ensuring reliable data transport.
7	Arduino IDE / PlatformIO	Used for developing, compiling, and deploying the C++/MicroPython firmware onto the ESP32 microcontrollers. Provides essential libraries for sensor interfacing and network communication.
8	Visual Studio Code	The primary integrated development environment for writing and managing the project's codebase, including the frontend, backend, and Python scripts.

Chapter 3

Software Requirement Specifications

3.1 Introduction

3.1.1 Definitions, Acronyms, and Abbreviations

Term	Definition
API	Application Programming Interface. A set of rules and protocols that allows different software applications to communicate with each other. In this project, it refers to the backend REST API that connects the user interface to the database and control logic.
ESP32	A series of low-cost, low-power system-on-a-chip microcontrollers with integrated Wi-Fi and dual-mode Bluetooth.
GUI	Graphical User Interface. A visual interface that allows users to interact with electronic devices through graphical icons and visual indicators.
HTTP	Hypertext Transfer Protocol. The protocol used for transmitting hypermedia documents, such as HTML. It is the foundation of data communication for the World Wide Web.
IDE	Integrated Development Environment. A software application that provides comprehensive facilities to computer programmers for software development.
MQTT	Message Queuing Telemetry Transport. A lightweight, publish-subscribe network protocol that transports messages between devices. Ideal for IoT applications.

Overview

This Software Requirement Specification (SRS) outlines the software requirements that form the basis of the Smart Irrigation System for Arecanut Farms. The system is designed to automate the irrigation process by using real-time data from soil moisture sensors and executing decisions based on predefined thresholds. The development of such a specification follows standard software

engineering practices to ensure clarity and completeness [45]. The goal of this SRS is to clearly define the functional and non-functional requirements of the system, establish a mutual understanding between developers and stakeholders, and serve as a reference for system design and testing.

The Smart Irrigation System software will:

- Monitor environmental conditions, especially soil moisture.
- Fetch water requirement thresholds based on soil, age, and season.
- Automatically control water delivery.
- Allow manual override through a user-friendly dashboard.
- Log events and provide real-time feedback to users.

The software components include a frontend web interface built using Next.js, a backend API developed in Node.js, a PostgreSQL database, an MQTT broker, and data analysis scripts written in Python.

3.2 General Description

This section provides a holistic overview of the "IoT-Based Smart Irrigation System," defining its position as a product, its core functionalities, the characteristics of its intended users, and the general constraints and assumptions that have fundamentally shaped its architecture and scope.

3.2.1 Product Perspective

The "IoT-Based Smart Irrigation System" is conceptualized as a complete, stand-alone product designed to function as an end-to-end solution for on-farm water management. It is not merely a software application but a fully integrated system that encompasses the entire data lifecycle, from physical sensing in the field to data-driven actuation and user-facing visualization. The product includes a distributed network of hardware sensors, a local processing gateway, and a comprehensive web-based dashboard. It is intended to be a modern replacement for, rather than an augmentation of, the traditional, labor-intensive methods of irrigation currently prevalent.

A key aspect of its product perspective is its operational independence. The system is architected to function as a self-contained unit, deliberately avoiding the need for integration into larger, potentially complex, pre-existing farm management software platforms. This design choice ensures simplicity in deployment and operation. The core philosophy is that the hardware (sensors, valves)

and software (control logic, database, web interface) work in seamless concert, with all critical operations and data storage handled locally on the farm. This on-premises approach is fundamental to its identity, ensuring that core functionalities remain robust and reliable, even in the event of intermittent or complete loss of internet connectivity. This perspective aligns with standard software engineering practices, where defining a clear system boundary is a crucial first step in managing project complexity and delivering a focused, effective solution .

3.2.2 Product Functions

The system is designed to deliver a specific set of high-value functions that directly address the core problems of manual irrigation. The successful execution of these functions defines the system's utility to the end-user. The primary product functions are as follows:

- **Real-Time Environmental Monitoring:** The system's foundational capability is its capacity to continuously collect live data from soil moisture sensors placed strategically within the arecanut farm. This provides an accurate, real-time assessment of the volumetric water content in the soil, replacing guesswork with empirical data.
- **Precision Volumetric Irrigation:** During every irrigation cycle, the system utilizes a water flow sensor to measure the exact volume of water being delivered. This moves beyond simple timed irrigation to ensure plants receive a precise, calculated quantity of water, which is critical for both optimal growth and resource conservation.
- **Automated, Rule-Based Control:** This is the intelligent core of the system. It automatically decides when to initiate and terminate irrigation based on a sophisticated but transparent set of rules. This logic considers the real-time soil moisture data in conjunction with static farm parameters like the age of the arecanut palms, the soil type, and the current month to make a context-aware decision.
- **Flexible Manual Override:** Acknowledging the farmer's expertise and the need for situational control, the system provides a simple and immediate manual override capability. Through the web dashboard, the farmer can bypass the automatic schedule to manually operate the irrigation pump at any time.
- **Data Visualization and System Monitoring:** The system presents all collected data through an intuitive graphical interface. Farmers can view live status updates, check historical trends through graphs and charts, and monitor the overall health and performance of the system, transforming raw data into actionable insights.

- **Historical Data Logging and Auditing:** Every critical event is automatically recorded. The system logs all sensor readings with timestamps and every control action (e.g., "Pump turned ON at 10:00 AM via AUTO mode"). This creates an invaluable historical log that can be reviewed for performance analysis, long-term trend identification, and record-keeping [46].

3.2.3 User Characteristics

The system is designed with a specific primary user in mind: The Farmer or Farm Manager. The profile of this user dictates the design philosophy for the entire user interface and operational workflow. It is assumed that this user is a subject matter expert in agriculture and arecanut cultivation but may not possess a deep technical background in software systems or electronics.

Therefore, the user's main priorities are reliability, ease of use, and the presentation of clear, actionable information. The system must not impose a steep learning curve. Consequently, the interface and operational logic are designed to be as intuitive as possible, requiring minimal training. Foundational principles of usability engineering, such as providing clear feedback, ensuring consistency, and offering a simple, uncluttered interface, have been central to the design process [47]. The only prerequisite technical skill is basic digital literacy, defined as the ability to operate a simple website on a standard computer, tablet, or smartphone. A secondary "Admin" role also exists, primarily for the initial setup and configuration of farm-specific details like soil type and crop planting dates.

3.2.4 General Constraints

The design and development of this project were guided and shaped by several significant, non-negotiable constraints that influenced technological choices and the overall scope.

- **Economic and Cost Constraints:** The system is explicitly intended for small to medium-scale farmers, a demographic that is often highly sensitive to cost. This economic reality mandated an architecture built upon affordable, open-source hardware (like Raspberry Pi and ESP32) and software. This constraint directly led to the avoidance of expensive, proprietary commercial-grade controllers and, crucially, the recurring subscription fees associated with commercial cloud platforms [48].

- **Operational Environment:** The system must be engineered for high reliability within a rural farm setting, where stable, high-speed internet connectivity cannot be guaranteed. This environmental constraint was a primary driver behind the architectural decision to make the system fully functional offline, ensuring that the core mission of irrigation continues uninterrupted.
- **User Interface Simplicity:** As defined by the user characteristics, the web dashboard had to be designed for ultimate simplicity, avoiding complex configuration menus or dense data tables in favor of clear visual indicators and straightforward controls.
- **Development Timeline:** The project was developed within the fixed and immovable timeframe of a single academic semester. This temporal constraint required a pragmatic focus on delivering the most critical core features for the initial prototype, while deferring more advanced functionalities to future development cycles.
- **Hardware Dependency:** The software is intrinsically linked to and dependent on the chosen set of hardware components. Its performance, accuracy, and reliability are directly contingent on the proper functioning, installation, and calibration of these physical devices.

3.2.5 Assumptions and Dependencies

The system's successful and reliable operation is predicated on a set of key assumptions about its environment and the validity of its inputs. The management of these factors is a critical aspect of risk mitigation in any IoT project [49].

- **Assumptions:**
 - **Correct Physical Installation:** It is assumed that all physical components sensors (soil moisture, water flow) and actuators (motorized valve) are installed in the correct locations as per operational guidelines to ensure meaningful data collection and effective water delivery.
 - **Sensor Accuracy:** The system operates on the assumption that the hardware sensors will provide readings that fall within their manufacturer-specified accuracy tolerances.
 - **Accurate Configuration Data:** The effectiveness of the rule-based logic is critically dependent on the accuracy of the initial configuration data provided by the user, such as the farm's soil type and the crop's planting date.

- **Stable Power Supply:** It is assumed that a continuous and stable power supply is available on the farm to run the Raspberry Pi gateway, the network equipment, and the water pump.
- **Dependencies:**
 - **Hardware Component Availability:** The project is dependent on the continued availability and proper functioning of the specific third-party hardware components selected for the prototype.
 - **Third-Party Software Libraries:** The software's functionality is dependent on the stability, correctness, and continued maintenance of the various open-source libraries and frameworks used in its construction (e.g., Node.js packages, Python libraries).
 - **Local Area Network (LAN):** The communication between the distributed sensor nodes (ESP32) and the central gateway (Raspberry Pi) is entirely dependent on a stable local Wi-Fi network that provides adequate signal coverage to all required points on the farm.

3.3 Functional Requirements

- **Module 1: Soil Moisture Monitoring**
 - **Introduction:** This module is responsible for the continuous and real-time measurement of soil moisture content. Its primary purpose is to provide the foundational data required to determine the immediate need for irrigation in a specific farm section.
 - **Input:** The system receives a raw analog voltage signal from the capacitive soil moisture sensor, which is directly proportional to the dielectric permittivity of the surrounding soil.
 - **Processing:** The firmware on the ESP32 microcontroller continuously reads the analog input signal. It converts this analog value into a calibrated digital moisture percentage. This processed data point is then packaged with a timestamp and a unique section identifier.
 - **Output:** The module outputs a data packet containing the moisture percentage, a precise timestamp, and the corresponding section ID. This packet is then published to a specific topic on the MQTT broker for collection by the central gateway.

- **Module 2: Water Requirement Calculation**

- **Introduction:** This module functions as the system's core intelligence engine. Its purpose is to dynamically calculate the ideal irrigation requirement for a given section of the farm by considering key agronomic and environmental variables.
- **Input:** The processing logic takes three parameters as input: the pre-configured soil type for the section, the current month of the year, and the calculated age of the arecanut plants (derived from a stored planting date).
- **Processing:** The system executes a lookup function on a predefined data structure named the Water_Requirement table. It uses the parameters (soil type, month, plant age) to decide this table and fetch the corresponding pre-determined value for the ideal water volume.
- **Output:** The module's output is the ideal water requirement, expressed in liters per day per palm. This value is then used by the Irrigation Control module as the target volume for automated irrigation cycles.

- **Module 3: Irrigation Control (Auto/Manual)**

- **Introduction:** This module serves as the command and control center for the physical irrigation hardware. It is responsible for managing the state of the motorized valve, operating either based on automated, data-driven logic or in response to direct manual commands from the user.
- **Input:** The module receives several inputs: the current operational mode (AUTO or MANUAL), the real-time soil moisture value from the monitoring module, the required water volume from the calculation module, and any direct user input from the web dashboard (e.g., "Turn On").
- **Processing:** The processing logic is conditional based on the operational mode:
 - In AUTO mode, the system continuously compares the incoming real-time moisture value against the calculated threshold. If the moisture is less than the threshold, it generates a command to activate the relay via the Raspberry Pi, thereby opening the valve.
 - In MANUAL mode, the system bypasses all automated logic and directly executes the command issued by the user, giving them priority control.

- **Output:** The primary output is a digital ON/OFF signal sent to the Wi-Fi relay controlling the motorized valve. Concurrently, every control action (valve opened, valve closed, mode switched) is recorded as a timestamped event in the system's control log for auditing and history.
- **Module 4: Water Flow Monitoring**
 - **Introduction:** This module is designed to measure the actual volume of water delivered during each irrigation event. Its purpose is to ensure irrigation accuracy, provide data for water usage analysis, and enable system diagnostics.
 - **Input:** The module receives a continuous stream of digital pulses from the Hall effect sensor within the YF-S201 water flow meter. The frequency of these pulses is directly proportional to the flow rate of the water.
 - **Processing:** The firmware on the ESP32 counts the number of incoming pulses over a defined interval to calculate the instantaneous flow rate in Liters/minute (L/min). This rate is then multiplied by the duration of the irrigation cycle to compute the total volume of water that has passed through the sensor.
 - **Output:** The final output is the total calculated volume of water delivered during the irrigation cycle, expressed in liters. This value is then logged to the database for accuracy tracking and to provide the farmer with precise water consumption data.

3.4 Non-Functional Requirements

- **Performance**

The end-to-end time, from a sensor reading being taken in the field to a corresponding control action (e.g., activating the valve) being executed by the gateway, shall not exceed 10 seconds. This ensures the system can respond in a timely manner to rapid changes in environmental conditions. Furthermore, the user interface must remain responsive, with data visualizations on the dashboard updating within 5 seconds of new data being received by the backend.

- **Scalability**

The software, running on the specified hardware (Raspberry Pi 4B), shall be capable of supporting up to 10 independent farm sections, each with its own configuration and control logic. The system must handle simultaneous data streams from up to 50 individual

sensors (e.g., 10 soil moisture sensors and 40 other potential sensors) without any discernible degradation in performance or violation of the 10-second latency requirement.

- **Accuracy**

This requirement is dependent on the hardware's capabilities, but the software must not introduce additional error. The system's firmware and backend must be able to process and log sensor readings within the following tolerances:

- Soil moisture readings shall have an accuracy of $\pm 5\%$ of the sensor's full-scale output.
- Water flow readings shall have an accuracy of $\pm 10\%$ of the measured value.

The software must correctly handle the calibration data to ensure the final values presented to the user and used in the logic are as accurate as possible.

- **Reliability**

The automated irrigation control logic (irrigation ON/OFF) must execute correctly 100% of the time based on the defined rules and real-time data. There should be no instances of the system failing to trigger a required action or triggering an action when conditions are not met. The core control loop running on the on-premises gateway should be designed for high availability, with an expected uptime of 99.9%, ensuring continuous operation even during internet outages.

- **Data Logging**

The system shall log all sensor readings (soil moisture, water flow) with their corresponding timestamps and section IDs. It must also log all significant control actions, including valve ON/OFF events, mode changes (Auto/Manual), and system startups. All logged data must be stored in the PostgreSQL database and be retrievable through the user interface for a minimum period of 30 days.

3.5 Design Constraints

The design and implementation of the IoT-Based Smart Irrigation System were governed by a specific set of constraints. These are non-negotiable limitations that have profoundly influenced architectural decisions, the selection of technology, and the overall scope of the project. Acknowledging these constraints is critical for understanding the rationale behind the final system design.

3.6.1 Standard Compliance

To ensure interoperability, reliability, and future maintainability, the system adheres to established industry and web standards.

- **Communication Protocols:**

- **Wireless Networking:** The local wireless communication between the ESP32 sensor nodes and the Raspberry Pi gateway is implemented in compliance with the IEEE 802.11n (Wi-Fi) standard. This ensures compatibility with a wide range of off-the-shelf networking hardware.
- **IoT Messaging:** The core data exchange protocol adheres to the MQTT (Message Queuing Telemetry Transport) v3.1.1 specification. This is an OASIS standard for lightweight, publish-subscribe messaging, chosen specifically for its efficiency in low-bandwidth and high-latency environments typical of IoT deployments [50].

- **Web Standards:**

- The user-facing web dashboard is developed in compliance with modern web standards, including HTML5, CSS3, and ECMAScript 2020+. This adherence ensures a consistent and accessible user experience across all major, standards-compliant web browsers without requiring special plugins or software.

3.6.2 Hardware Limitations

The strategic choice to use affordable, consumer-grade hardware imposes several key limitations that directly shaped the software's design and capabilities.

- **Processing and Memory Resources:** The system relies on the computational resources of a Raspberry Pi 4 and ESP32 microcontrollers. While highly capable for their intended tasks, these devices possess finite processing power and RAM compared to enterprise-grade servers. This fundamental hardware constraint precludes the implementation of computationally expensive algorithms, such as real-time machine learning or deep learning models, within the current prototype. Consequently, the software design must be lightweight and optimized for efficient memory and CPU usage to ensure smooth, responsive operation [51].
- **Sensor Accuracy and Locality:** The use of low-cost capacitive soil moisture sensors introduces two primary limitations that the software must account for:

- **Localized Measurement:** Each sensor provides a point-in-time measurement for a very small, localized volume of soil. It cannot represent the average moisture level across an entire field, which can have significant variations. This physical limitation is a direct constraint on the system's scope, making single-zone irrigation control the only viable approach for the prototype [52].
- **Accuracy Tolerance:** The sensors operate with a manufacturer-specified accuracy tolerance (e.g., $\pm 5\%$). The software must be designed to be robust in the face of this inherent imprecision. It must handle minor data fluctuations and potential noise without triggering erratic or unstable behavior in the control loop. This involves data smoothing techniques and logic that is not overly sensitive to minute changes [53].
- **Wireless Communication Range:** The system's dependence on Wi-Fi for communication between the sensor nodes and the gateway is a significant environmental constraint. The effective range of a standard Wi-Fi signal is limited and can be adversely affected by distance, physical obstacles (such as dense foliage, buildings, or terrain changes), and radio frequency interference from other devices. This constrains the physical size of the agricultural area that can be reliably covered by a single gateway without requiring additional networking hardware like Wi-Fi extenders or a more complex mesh network.

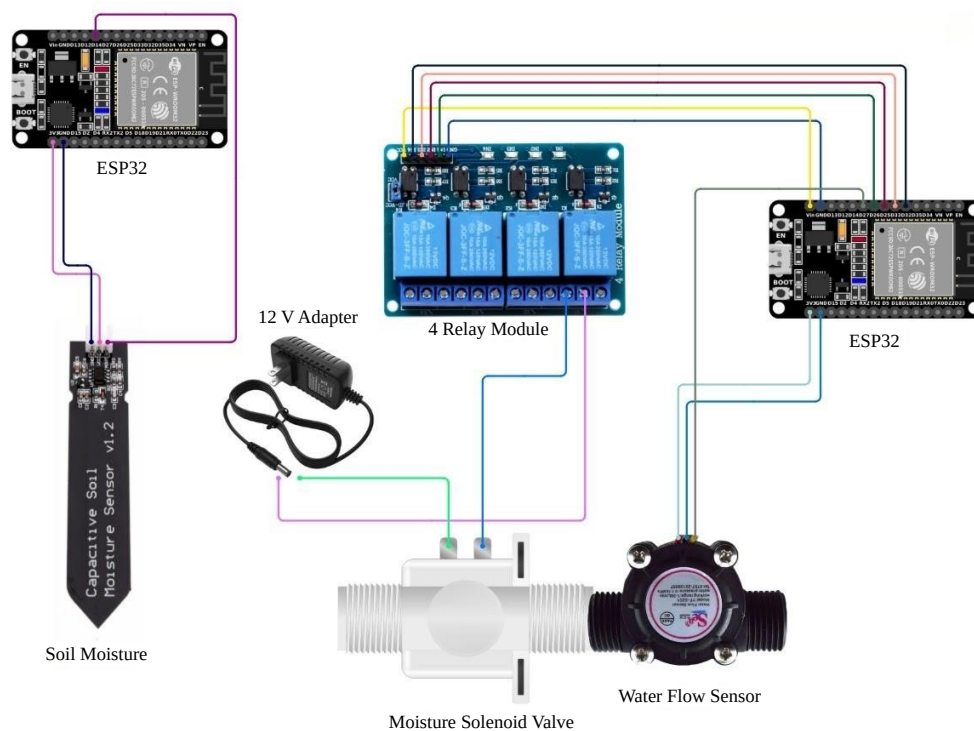


Figure 3.1 Circuit Connection of the Hardware Components

Chapter 4

System Design

4.1 System Perspective

4.1.1 Problem Specification

The foundational step in designing any effective system architecture is to translate the high-level project goals into a set of specific, technical problems that the architecture must solve. From a system design viewpoint, the core challenge of this project is to create a cohesive, end-to-end system that effectively bridges the physical world of the farm with the digital world of data processing and control. The architecture must directly address the following well-documented problems inherent in traditional arecanut farming.

- **Massive Water Wastage:** This is the most visible problem stemming from conventional methods like flood irrigation or fixed-timer sprinklers. These approaches are inherently imprecise and almost always result in the application of more water than the plants actually need. This excess is lost to evaporation or deep percolation below the root zone, wasting a precious natural resource and increasing operational costs through higher energy consumption for pumping. The proposed architecture must solve this by enabling a closed-loop system where irrigation is triggered by data, not by a clock. The system must also be designed with hardware that provides accurate measurements; for instance, using capacitive soil moisture sensors, which offer superior accuracy and longevity compared to their resistive counterparts, is a key architectural decision [54].
- **Compromised Crop Health and Reduced Yield:** Inefficient watering directly harms the arecanut palms in two critical ways:
 - Over-irrigation leads to waterlogged soil, suffocating the roots and creating anaerobic conditions ideal for fungal diseases like root rot. It also leaches essential nutrients from the soil, diminishing its fertility. The system architecture must prevent this by ensuring irrigation cycles are terminated precisely when the soil reaches its optimal moisture capacity.
 - Under-irrigation is equally damaging. A farmer's misjudgment or a rigid schedule that fails to account for a sudden heatwave can cause severe moisture stress. This stunts palm growth, reduces nut size and quality, and ultimately lowers the farm's yield. The

architecture must be responsive enough to detect these deficit conditions in near real-time and initiate a corrective watering cycle. The development of the system's firmware, often streamlined using toolchains like PlatformIO, is critical to ensuring this responsiveness .

- **Dependency on Manual Labor:** Traditional irrigation is incredibly labor-intensive, requiring farmers to be physically present to operate pumps and valves. This constant manual intervention is time-consuming, physically demanding, and introduces a significant risk of human error a forgotten pump can flood a field, while a delay can cause crop stress. The architecture must solve this by providing reliable automation, freeing the farmer from these repetitive tasks. The design of this automated system must adhere to well-defined specifications to ensure it is robust and dependable, following standards like those outlined by the IEEE for software requirements [55].
- **Complete Lack of Visibility and Data:** Perhaps the most significant underlying issue is that farmers operate "in the dark," lacking the real-time data needed to make informed decisions. They cannot know the precise moisture level in the soil without manual inspection and have no historical records to analyze trends or optimize water usage. This lack of visibility makes it impossible to detect subtle problems like a slow pipe leak. The system architecture must solve this by creating a persistent data log and presenting this information through a clear, accessible interface. The elicitation of these non-functional requirements what the user needs to *see* and *know* is a critical part of the design process [56].

4.1.2 Block Diagram

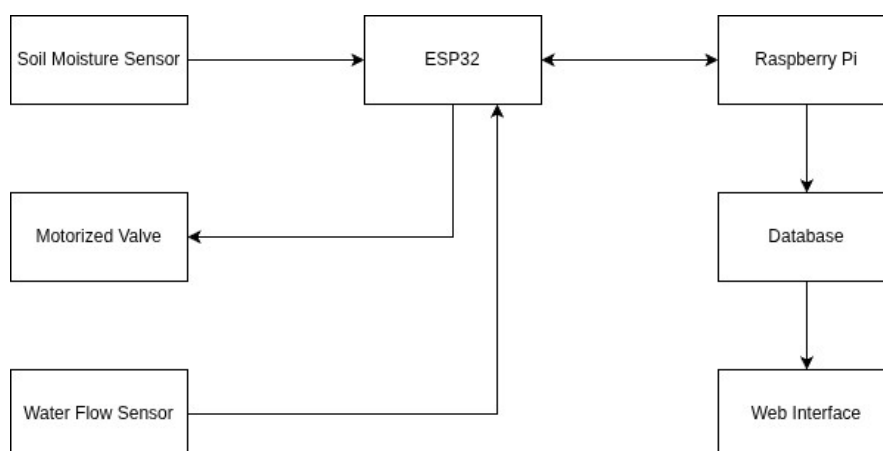


Figure 4.1 Representation of Block Diagram of IoT based Smart Irrigation in Arecanut Farms

The figure 4.1 illustrates a complete data and control loop, beginning with the on-field Soil Moisture and Water Flow Sensors which gather real-time environmental data. This information is collected by the ESP32, a local controller that transmits the data to the Raspberry Pi, the system's central gateway. Acting as the brain, the Raspberry Pi processes this incoming data, applies a rule-based logic to determine if irrigation is required, and logs all events and sensor readings into the Database. If action is needed, the Raspberry Pi sends a command back to the ESP32, which in turn controls the Motorized Valve to start or stop the flow of water. Finally, the Web Interface retrieves historical and live data from the database, presenting a comprehensive overview to the farmer and providing a means for manual control, thus completing the human-in-the-loop system.

4.1.3 Data Definition

users

Field Name	Data Type	Description
id	String	Primary key, cuid()
email	String	Unique email address
password	String	Hashed password
name	String (Optional)	Optional user name
farm_ids	Integer[]	Array of farm IDs associated with user

farms

Field Name	Data Type	Description
id	Integer	Primary key, Auto-increment
name	String	Name of the farm
location	String (Optional)	Optional location info

sections

Field Name	Data Type	Description
id	Integer	Primary key, Auto-increment
name	String	Name of the section
farm_id	Integer	Foreign key → farms(id)
section_number	Integer	Unique number (1-4) within a farm

moisture_readings

Field Name	Data Type	Description
id	Integer	Primary key, Auto-increment
farm_id	Integer	Foreign key → farms(id)
section_number	Integer	Refers to section number within farm
value	Float	Moisture reading value
timestamp	DateTime	Timestamp of the reading

irrigation-events

Field Name	Data Type	Description
id	Integer	Primary key, Auto-increment
farm_id	Integer	Foreign key → farms(id)
section_number	Integer	Section within the farm
water_ml	Float	Amount of water used in milliliters
start_time	DateTime	When irrigation started
end_time	DateTime	When irrigation ended

moisture_device_status

Field Name	Data Type	Description
id	Integer	Primary key
device_id	String	Identifier of the device
farm_id	Integer	Foreign key → farms(id)
section_number	Integer	Section within the farm
mqtt	Boolean	MQTT connectivity status
wifi	Boolean	Wi-Fi status
uptime	BigInt	Uptime in seconds
timestamp	DateTime	Time of status capture
last_error	String	Description of last error
enable_deep_sleep	Boolean	Whether deep sleep is enabled
reporting_interval	Integer	Reporting interval in seconds
deep_sleep_duration	Integer	Duration of deep sleep in seconds
createdAt	DateTime	Creation timestamp

irrigation_device_status

Field Name	Data Type	Description
id	Integer	Primary key
device_id	String	ID of the irrigation device
farm_id	Integer	Foreign key → farms(id)
section_number	Integer	Section within the farm
uptime	BigInt	Uptime in seconds
wifi	Integer	Wi-Fi signal strength
mqtt	Integer	MQTT signal strength
last_error	String	Last error reported
valve_on	Integer	Status of valve (0 or 1)
mode	String	Operating mode (manual/auto/etc.)
latest_moisture	Integer	Last moisture reading
min_threshold	Integer	Moisture lower limit (default: 30)
max_threshold	Integer	Moisture upper limit (default: 70)
pulse_count	Integer	Water pulses recorded
water_ml	Integer	Water dispensed in milliliters
timestamp	DateTime	Status time

device_acks

Field Name	Data Type	Description
id	Integer	Primary key
device_id	String (Optional)	Device ID (optional)
farm_id	Integer	Foreign key → farms(id)
section_number	Integer	Section number
ack_json	Json	Acknowledgement message in JSON format
timestamp	DateTime	Time when ack was received

4.1.4 Module Specification

Module 1: Soil Moisture Monitoring

- **Purpose:**

To continuously acquire real-time soil moisture data from the physical environment. This module serves as the primary sensory input for the entire system, providing the critical data point needed to assess the current irrigation need of a farm section.

- **Inputs:**

- A raw analog voltage signal generated by the capacitive soil moisture sensor, which corresponds to the soil's dielectric constant.
- A unique device identifier (section_device_id) to associate the reading with a specific sensor.

- **Functional Description:**

The firmware running on the ESP32 microcontroller is responsible for the functions of this module. It periodically reads the analog signal from the sensor's output pin. This raw value is then converted into a calibrated, human-readable moisture percentage using a predefined conversion formula. The resulting percentage is packaged with a precise timestamp to create a complete data record.

- **Outputs:**

- A structured data packet containing the calculated moisture percentage, a timestamp, and the section_device_id. This packet is published over the local network to a designated MQTT topic for consumption by the Irrigation Control module.

Module 2: Irrigation Control (Auto/Manual)

- **Purpose:**

To serve as the central decision-making and command execution core of the system. It is responsible for controlling the state of the motorized irrigation valve based on either automated, data-driven logic or direct user commands.

- **Inputs:**

- The current operational mode of the system (AUTO or MANUAL).
- A continuous stream of real-time moisture data packets from the Soil Moisture Monitoring module.

- The configured operational parameters (e.g., moisture threshold) provided by the Water Requirement Calculation module.
- User-initiated commands received from the web interface (e.g., "Manual ON," "Switch to AUTO").
- **Functional Description:**

This module runs on the Raspberry Pi gateway and operates conditionally based on the system's mode:

 - In AUTO mode, it continuously listens for moisture data on the MQTT topic. For each incoming message, it compares the moisture value against the configured threshold. If the moisture is below the threshold, it generates and dispatches a command to open the valve.
 - In MANUAL mode, the threshold-based logic is suspended. The module listens for commands originating from the user interface and immediately relays them to the valve actuator, giving the user priority control.
- **Outputs:**
 - A digital ON/OFF command signal transmitted to the Wi-Fi relay that controls the motorized valve.
 - A corresponding event log (e.g, valve_status: ON, mode: AUTO, timestamp: ...) which is sent to the database for permanent storage.

Module 3: Water Flow Monitoring

- **Purpose:**

To accurately measure and record the total volume of water delivered during each irrigation cycle. This provides crucial data for water management analytics and serves as a verification mechanism for the system's operation.
- **Inputs:**
 - A stream of digital pulses generated by the YF-S201 Hall effect water flow sensor, where the pulse frequency is proportional to the water's flow rate.
- **Functional Description:**

The ESP32 firmware continuously counts the incoming pulses from the flow sensor. By measuring the number of pulses over a fixed time interval, it calculates the instantaneous flow rate in Liters per Minute (L/min). During an active irrigation cycle, the firmware

integrates this flow rate over time to compute the total accumulated volume of water that has been dispensed.

- **Outputs:**
 - The final calculated water volume, in liters, for the completed irrigation cycle. This data is published via MQTT and subsequently logged in the database for historical analysis and accuracy tracking.

4.1.5 Assumptions made

- **Soil moisture sensors are pre-calibrated for accuracy.**

This assumption presumes that the sensors are not simply "plug and play." Calibration is the crucial process of teaching the sensor what its raw electrical readings actually mean in the context of the specific farm soil. It involves testing the sensor's output in soil with known moisture levels (e.g., completely dry, perfectly damp, fully saturated) and adjusting the software's conversion formula so that it translates these raw values into accurate, human-readable percentages. We are assuming this essential setup step has been completed before the system begins its automated operation.

- **Internet connectivity is available for communication.**

This is a nuanced assumption. While the core, real-time irrigation control loop (sensor -> Raspberry Pi -> valve) is designed to function offline, this assumption applies to any feature that requires communication with the outside world. This primarily includes the initial setup of the system (which requires downloading software libraries and packages) and any potential for remote monitoring or software updates in the future.

- **Each field section has a unique ID and fixed configuration.**

This assumption dictates how the farm's data is organized. It means that every distinct irrigation zone managed by the system is assigned a unique identifier (e.g., "Section_A"). All the crucial information for that zone such as its soil type, the planting date of its crops, and the specific sensors assigned to it is tied to this ID in the database. This configuration is considered static during normal operation.

- **The user interacts with the system via web interface only.**

The only officially supported method for a user to monitor data and control the system is through the web-based dashboard, accessed via a standard web browser on a device like a computer, tablet, or smartphone.

- **Farmers can understand basic UI/UX elements (dashboard usability).**

We are assuming that the end-user possesses a basic level of digital familiarity. This includes recognizing and understanding common web elements like buttons, menus, and data visualizations such as graphs and status indicators (e.g., a green light for "ON").

- **The valve, relay module, and flow sensor are correctly installed and functional.**

This presumes that the physical hardware layer has been set up correctly and is fully operational. This includes ensuring the motorized valve is properly fitted into the plumbing without leaks, the relay module is safely and correctly wired to both the power source and the valve, and the water flow sensor is installed in the proper orientation to measure flow accurately.

4.2 Context Diagram

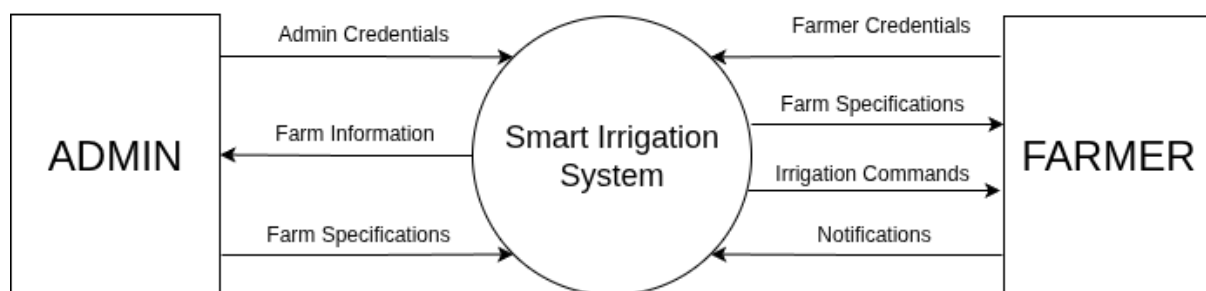


Figure 4.2 Context Diagram of Smart Irrigation

4.2.1 External Entities

- **ADMIN:** The administrator is responsible for the overall management and setup of the system. This user has high-level privileges, such as creating farmer accounts and configuring new farms.
- **FARMER:** The farmer is the primary end-user of the system. Their interaction is focused on monitoring their specific farm and controlling its irrigation operations.

4.2.2 The Central Process

- **Smart Irrigation System:** This central circle represents the entire software application. It encapsulates all the modules, logic, and functions described previously, including user authentication, data processing, rule-based control, and data logging. From this diagram's perspective, we are not concerned with the internal workings, only with what data goes in and what data comes out.

4.2.3 Data Flows (Inputs and Outputs)

- **Inputs (Data flowing INTO the Smart Irrigation System):**
 - **From the ADMIN:**
 - Admin Credentials: This is the username and password the administrator provides to log in and gain authenticated access to the system.
 - Farm Specification: This represents the initial setup data that the admin inputs. This includes creating new farmer profiles, registering a new farm, and defining its core properties like soil type, location, and size.
 - **From the FARMER:**
 - Farmer Credentials: This is the username and password the farmer uses to log into their specific account on the dashboard.
 - Irrigation Commands: (Note: While the arrow in the diagram points from the system to the farmer, the logical flow for a command is from the user into the system). This represents the manual control actions initiated by the farmer, such as clicking a button to turn the pump ON/OFF or switching the system from AUTO to MANUAL mode.
- **Outputs (Data flowing FROM the Smart Irrigation System):**
 - **To the ADMIN:**
 - Farm Information: This represents the data that the system provides back to the administrator. This could include a list of all registered farms, system health status, or high-level reports on water usage across the platform.
 - **To the FARMER:**
 - Farm Specifications: The system displays the configured details of the farm (its name, soil type, etc.) to the farmer on the dashboard for their information.
 - Notifications: These are alerts and real-time status updates that the system sends to the farmer's dashboard. Examples include "Irrigation Started in North Block," "Soil Moisture is at 55%," or "Manual Mode Activated."

Chapter 5

Detailed Design

5.1 System Design

- **Intuitive Real-World Mapping:**

Object-Oriented Analysis excels at creating an intuitive mapping between the real-world entities of a problem and the software components that represent them. The domain of a smart irrigation system naturally consists of tangible and conceptual "things" that can be directly modeled as objects. For example, a Farm is not just a name; it is an entity with specific attributes like location, totalArea, and soilType, and it has behaviors, such as the ability to addSection() or getDetails(). Similarly, a Farmer is an object with properties (name, email) and actions (login(), viewDashboard()). Even hardware components like a SoilMoistureSensor and a MotorizedValve are cleanly represented as objects, each with its own state (e.g., currentReading, isOpen) and methods (readMoisture(), openValve()).

- **Encapsulation and Modularity:**

A core tenet of OOA is encapsulation, which involves bundling an object's data (attributes) and the methods that operate on that data together into a single, self-contained unit. This creates a "black box" effect, where the internal complexity of an object is hidden from the rest of the system. For instance, the SoilMoistureSensor object is responsible for its own internal workings. The rest of the application does not need to know how it reads moisture—whether it involves converting an analog voltage, communicating over an I2C bus, or parsing a complex data string. It simply calls the public readMoisture() method and trusts that it will receive a valid percentage value in return.

- **Scalability and Reusability through Inheritance and Polymorphism:**

The object-oriented design makes the system inherently scalable and built for future growth. OOA provides powerful mechanisms like inheritance and polymorphism that promote code reuse and simplify the process of extending the system. For example, we could define a general Device class that contains common attributes and methods for all hardware components, such as a deviceID, a status property, and methods like getStatus() or runDiagnostics().

5.1.1 Class Diagram

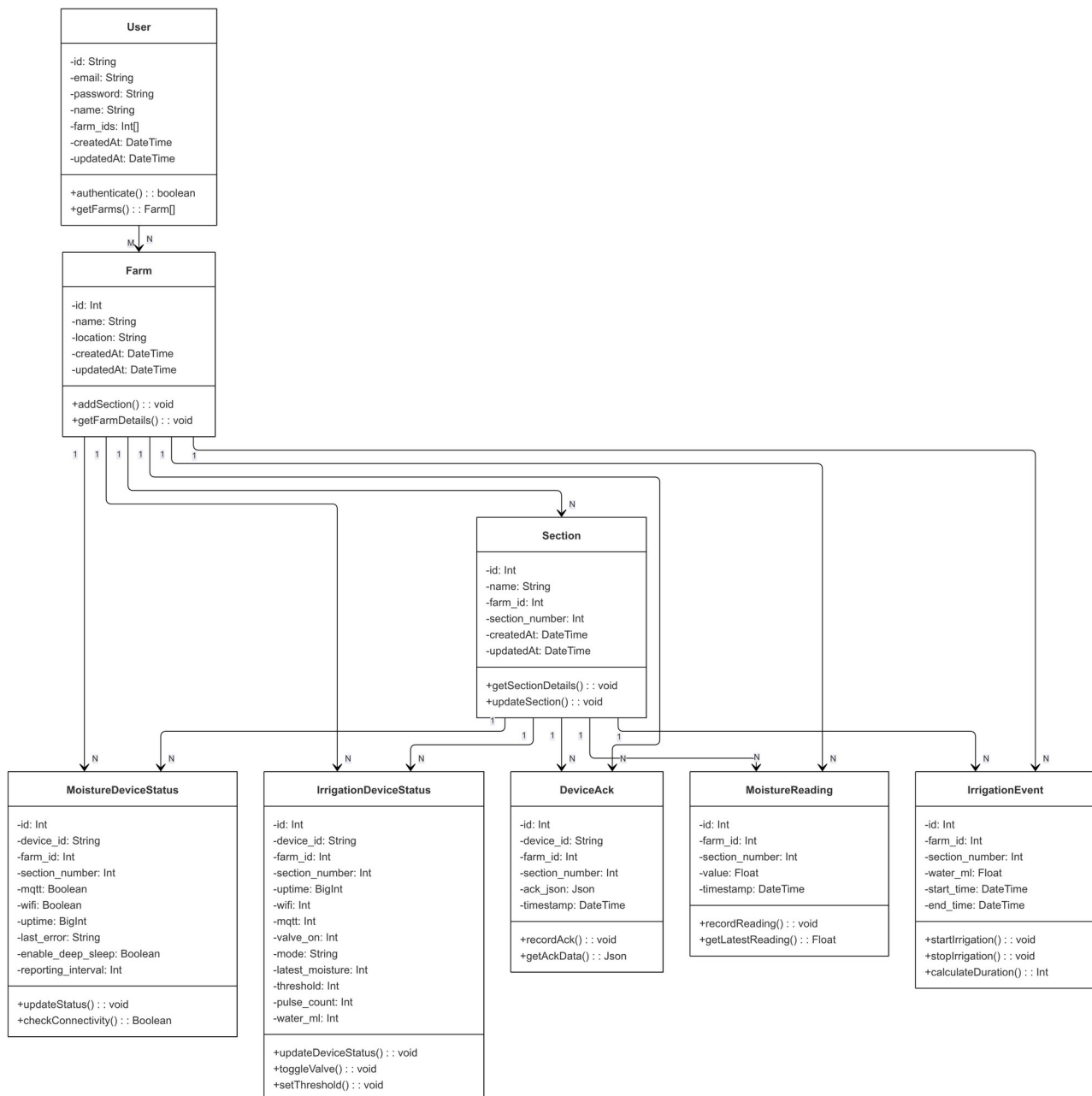


Figure 5.1 Representation of Class Diagram of IoT Based Smart Irrigation

This figure 5.1 shows an IoT smart irrigation system where a User manages Farms, each divided into Sections with connected devices. **MoistureDeviceStatus** and **IrrigationDeviceStatus** track sensor readings, connectivity, and valve control. **MoistureReading** logs soil data, while **IrrigationEvent** records watering activities. **DeviceAck** stores device confirmations, enabling full monitoring and control of irrigation processes.

5.1.2 Use-Case Diagram

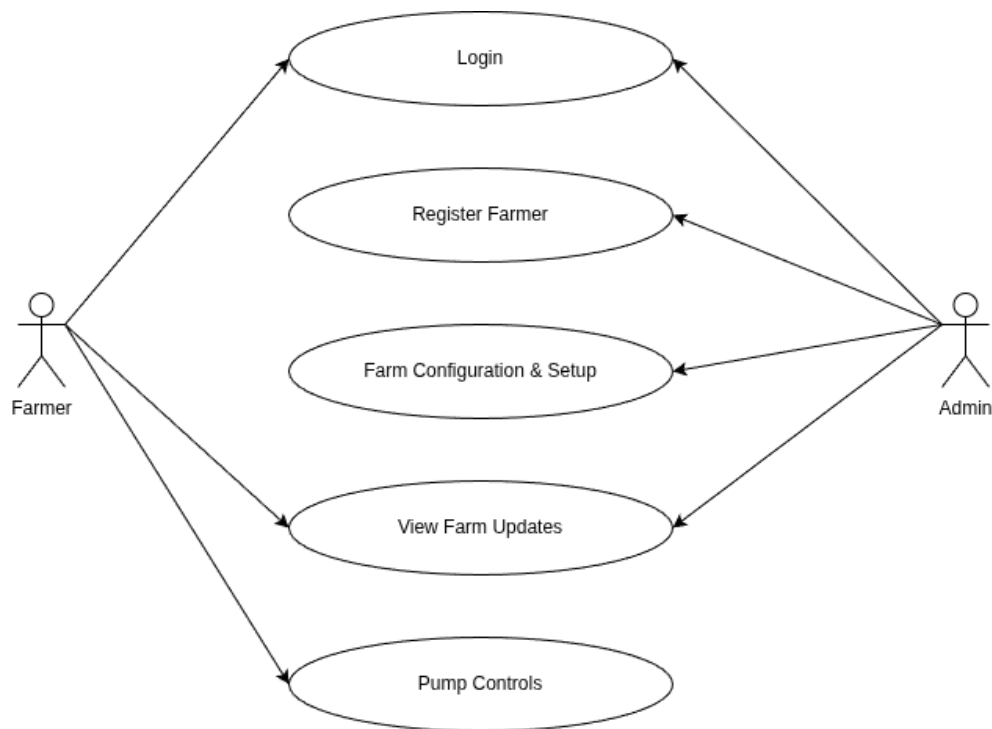


Figure 5.2 Representation of Use-Case Diagram for IoT based Smart Irrigation

In figure 5.2 Both the Farmer and Admin share common functionalities like Login to access the system and View Farm Updates to monitor its status. However, the Farmer has exclusive access to operational functions like Pump Controls, while the Admin is solely responsible for administrative tasks such as Register Farmer and the initial Farm Configuration & Setup. This clearly separates the day-to-day operational use from the administrative setup of the system.

5.1.3 Activity Diagram

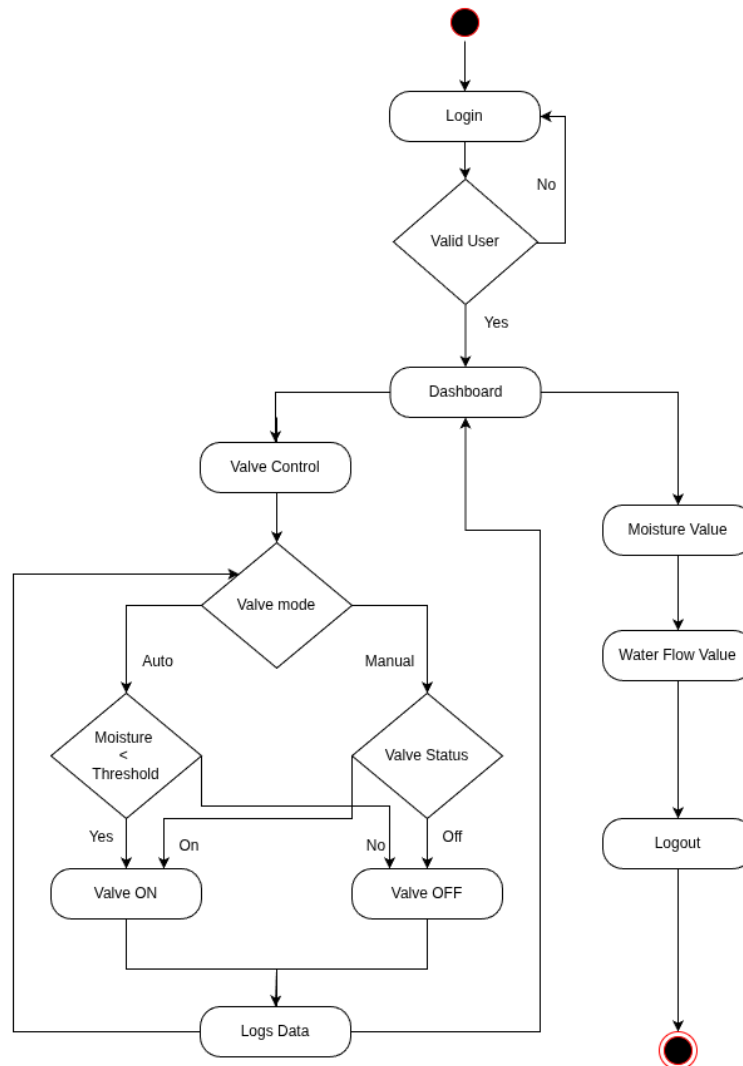


Figure 5.3 Representation of Activity Diagram of Smart Irrigation

Figure 5.3 The diagram clearly shows the system's dual-mode logic: if the "Valve mode" is Auto, the system's state is determined by comparing the moisture level to a threshold. If the mode is Manual, the user directly controls the valve's ON/OFF state. All control actions converge on the Logs Data step, ensuring every event is recorded before the workflow returns to the dashboard or ends with a logout.

5.1.4 Sequence Diagram

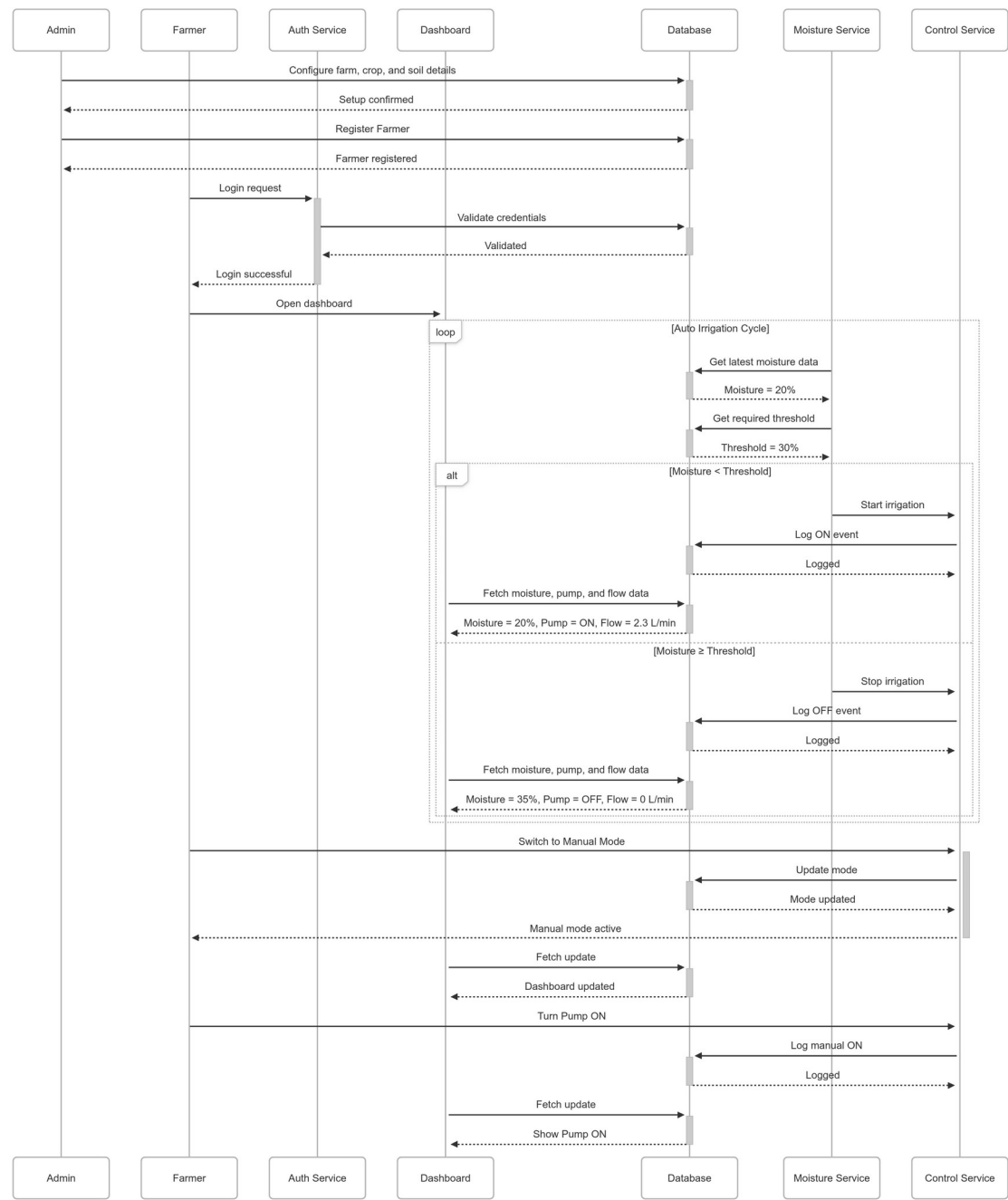


Figure 5.4 Sequence Diagram for Smart Irrigation

This Figure 5.4 illustrates the time-ordered interactions between users and the system's microservices for key scenarios like user login, farm setup, and irrigation control. It clearly visualizes the two primary operational modes: the Auto Irrigation Cycle, where the system automatically checks moisture against a threshold to control the valve, and the Manual Mode, where a farmer's command directly triggers the pump.

5.1.5 Data Flow Diagrams

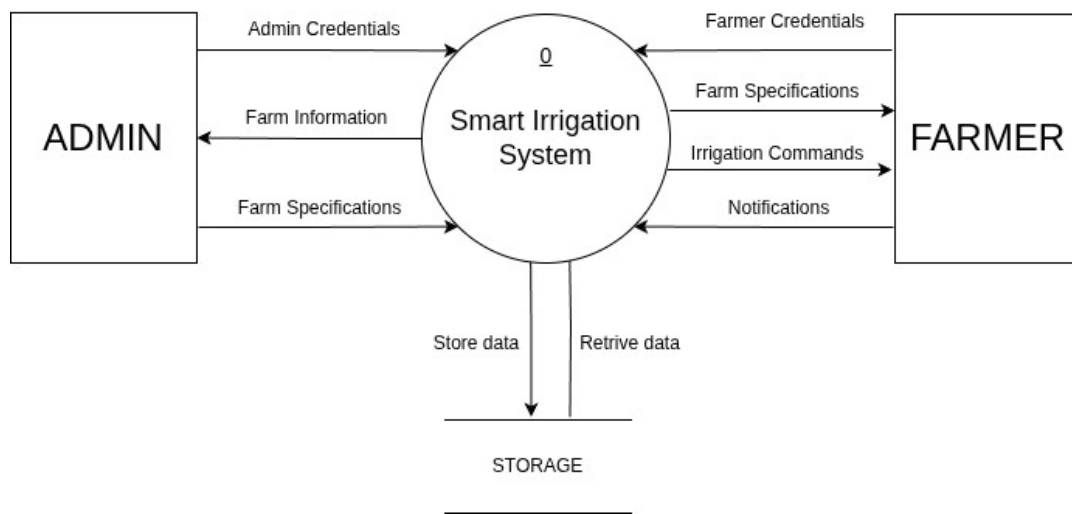


Fig 5.5 Data Flow Diagram of Smart Irrigation – Level 0

This Level 0 Data Flow Diagram, also known as a Context Diagram, provides a high-level overview of the entire Smart Irrigation System as a single, central process. It clearly defines the system's boundaries by illustrating how it exchanges data with external entities: the ADMIN for setup and configuration, and the FARMER for operational monitoring and control.

The diagram also shows that the system has a fundamental interaction with a STORAGE entity, where it continuously stores all operational data and from which it retrieves information to present back to the users.

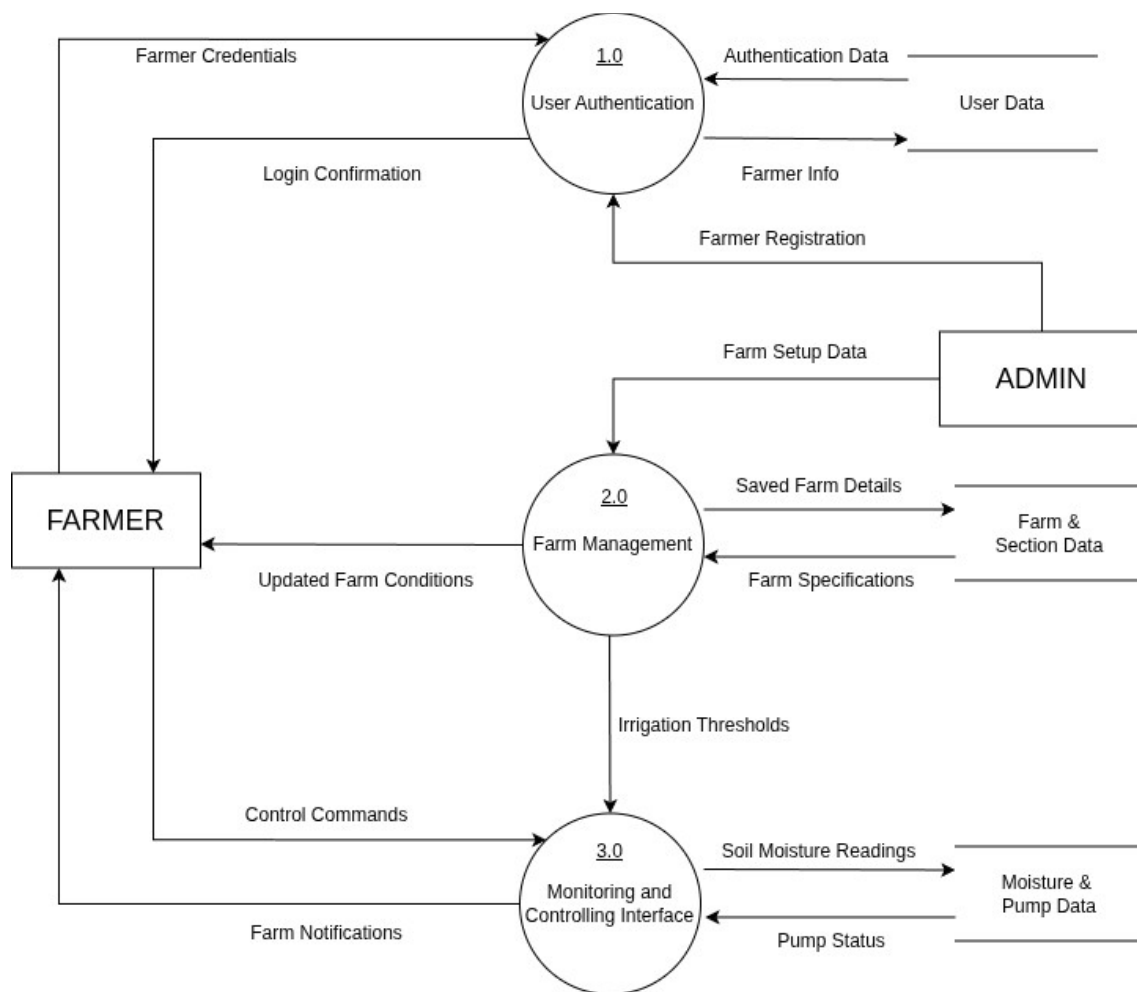


Figure 5.6 Data Flow Diagram of Smart Irrigation – Level 1

This Level 1 Data Flow Diagram expands upon the context diagram, decomposing the main system into three core processes: 1.0 User Authentication, 2.0 Farm Management, and 3.0 Monitoring and Controlling Interface. It illustrates how data flows between these processes, the external actors (FARMER, ADMIN), and various data stores.

The diagram clearly separates the system's functions: handling user access (1.0), configuring farm details and irrigation thresholds (2.0), and managing the real-time operational loop of monitoring sensor data and executing control commands (3.0).

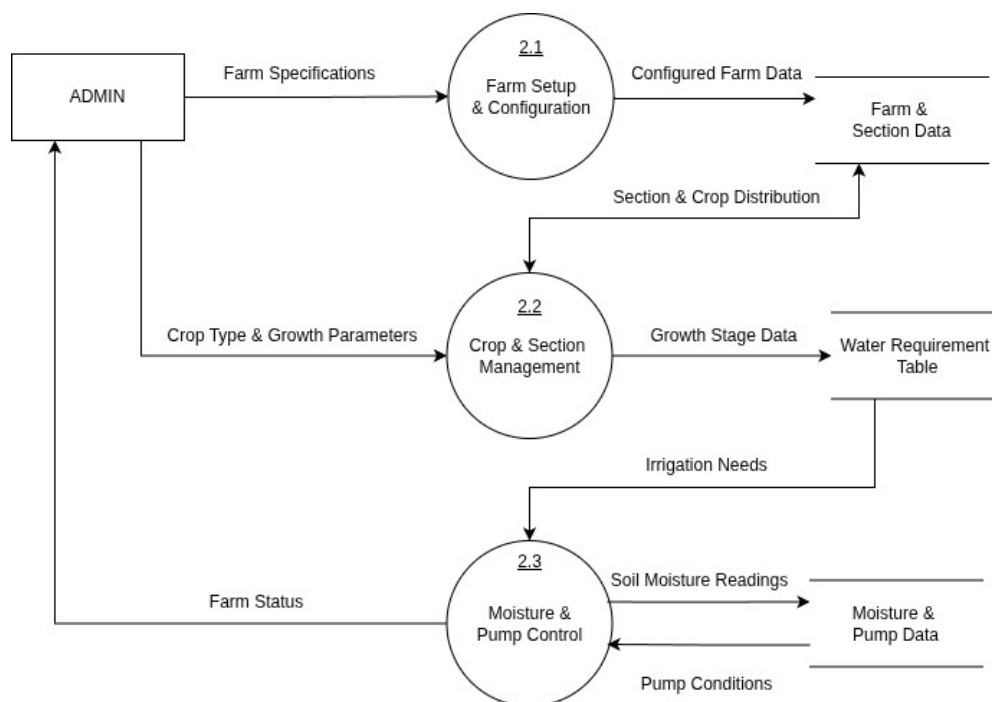


Figure 5.7 Data Flow Diagram of Smart Irrigation – Level 2

This Level 2 Data Flow Diagram offers a granular view of the administrative and control processes within the system. It begins with the ADMIN providing initial Farm Specifications to the Farm Setup & Configuration (2.1) process, which establishes the basic farm and section data. Following this, the Crop & Section Management (2.2) process uses specific Crop Type & Growth Parameters and consults the Water Requirement Table to determine the precise Irrigation Needs. These needs, which represent the core irrigation rules, are then fed into the Moisture & Pump Control (2.3) process. This operational module takes live Soil Moisture Readings, applies the rules to determine the necessary Pump Conditions, and executes the control actions. The resulting Farm Status is then reported back to the ADMIN, completing the high-level monitoring and control loop.

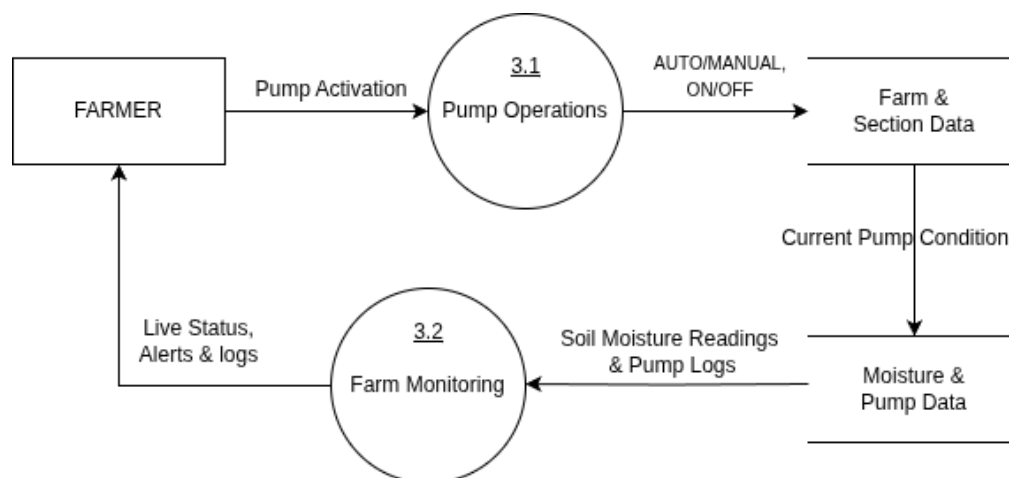


Figure 5.8 Data Flow Diagram of Smart Irrigation – Level 2

This Level 2 Data Flow Diagram details the system's operational loop from the FARMER's perspective, breaking it into control and monitoring functions. When the farmer initiates a Pump Activation command, it is handled by the Pump Operations (3.1) process, which then updates the system's current state (e.g., AUTO/MANUAL, ON/OFF) in the Farm & Section Data store.

In parallel, the Farm Monitoring (3.2) process continuously gathers real-time Soil Moisture Readings & Pump Logs from the data stores. It synthesizes this live data with the current pump status set by the operations module. Finally, this consolidated information is presented back to the FARMER as Live Status, Alerts & logs, creating a complete and interactive feedback loop.

5.2 Detailed Design

5.2.1 Design Decision

Several critical design decisions were made to ensure the system is efficient, maintainable, and robust.

- **Data Transmission Format:**
 - A lightweight JSON (JavaScript Object Notation) format was selected for all MQTT message payloads between the ESP32 nodes and the Raspberry Pi gateway.
 - JSON is human-readable, which greatly simplifies debugging. It is also natively supported by virtually all modern programming languages (including Python and JavaScript/Node.js), eliminating the need for complex parsing libraries. Its text-based

format has a slightly higher overhead than a pure binary format but offers far greater flexibility and ease of development.

- **Control Logic Approach:**

- A procedural, state-driven approach was chosen for the main control loop running on the Raspberry Pi. The system maintains a simple state (e.g, Manual ,Automatic) and makes decisions based on the current state and incoming sensor data.
- This approach is transparent, easy to debug, and highly reliable. While more complex object-oriented or event-driven models could be used, a procedural state machine is perfectly suited for the system's requirements and avoids unnecessary complexity. It ensures that the system's behavior is predictable and deterministic.

- **Database Interaction:**

- All database write operations (logging sensor data and events) are designed to be asynchronous. The main control loop does not wait for a database write to complete before proceeding with its next task.
- This prevents the primary control loop from being blocked by potentially slow I/O operations. If the database is temporarily slow or busy, the real-time responsiveness of the irrigation control is not compromised, enhancing the overall reliability of the system.

Chapter 6

Implementation

6.1 Code Snippets

6.1.1 MQTT Services Configuration

```
// MQTT Service Configuration
// Add or import your mqttConfig object here
const mqttConfig = {
  brokerUrl: "mqtt://your-broker-url",
  username: "your-username",
  password: "your-password",
  reconnectPeriod: 1000,
  connectTimeout: 30 * 1000,
};

export class MqttService {
  private client: mqtt.MqttClient | null = null;

  public async connect(): Promise<void> {
    this.client = mqtt.connect(mqttConfig.brokerUrl, {
      username: mqttConfig.username,
      password: mqttConfig.password,
      reconnectPeriod: mqttConfig.reconnectPeriod,
      connectTimeout: mqttConfig.connectTimeout,
      clean: true,
      keepalive: 60,
      // MQTTS configuration
      rejectUnauthorized: false, // Allow self-signed certificates
    });
  }

  // Subscribe to IoT device topics
  private subscribeToTopics(): void {
    const topics = [
      "farm/+/section/+/moisture",
      "farm/+/section/+/status",
      "farm/+/section/+/irrigation",
      "farm/+/section/+/ack",
      "farm/+/section/+/config",
      "farm/+/test",
    ];
  }
}
```


6.1.2 Moisture Data Handling

```
export async function handleMoistureMessage(
  farmId: number,
  sectionNumber: number,
  data: any
): Promise<void> {
  // Save moisture reading to database
  const serverTimestamp = new Date();
  await prisma.moistureReading.create({
    data: {
      farm_id: farmId,
      section_number: sectionNumber,
      value: parseFloat(data.value),
      timestamp: serverTimestamp,
    },
  });

  // Broadcast real-time update
  broadcastSectionUpdate(farmId, sectionNumber, {
    moisture_value: parseFloat(data.value),
    timestamp: serverTimestamp.toISOString(),
    type: "moisture_update",
  });
}
```

6.1.3 Automated Irrigation Control

```
router.post(
  `/:farmId/:sectionNumber/irrigate`,
  authenticateToken,
  validateFarmAccess("params"),
  async (req: Request, res: Response) => {
    const { farmId, sectionNumber } = req.params;
    const { duration = 60 } = req.body;

    try {
      // Send irrigation command via MQTT
      await mqttService.sendIrrigationCommand(
        parseInt(farmId),
        parseInt(sectionNumber),
        duration
      );

      // Update database
      await prisma.irrigationEvent.create({
        data: {
          farm_id: parseInt(farmId),
          section_number: parseInt(sectionNumber),
          water_ml: 0, // Will be updated by device
          start_time: new Date(),
          end_time: new Date(Date.now() + duration * 1000),
        },
      });
    }
  });
```

6.2 Implementation

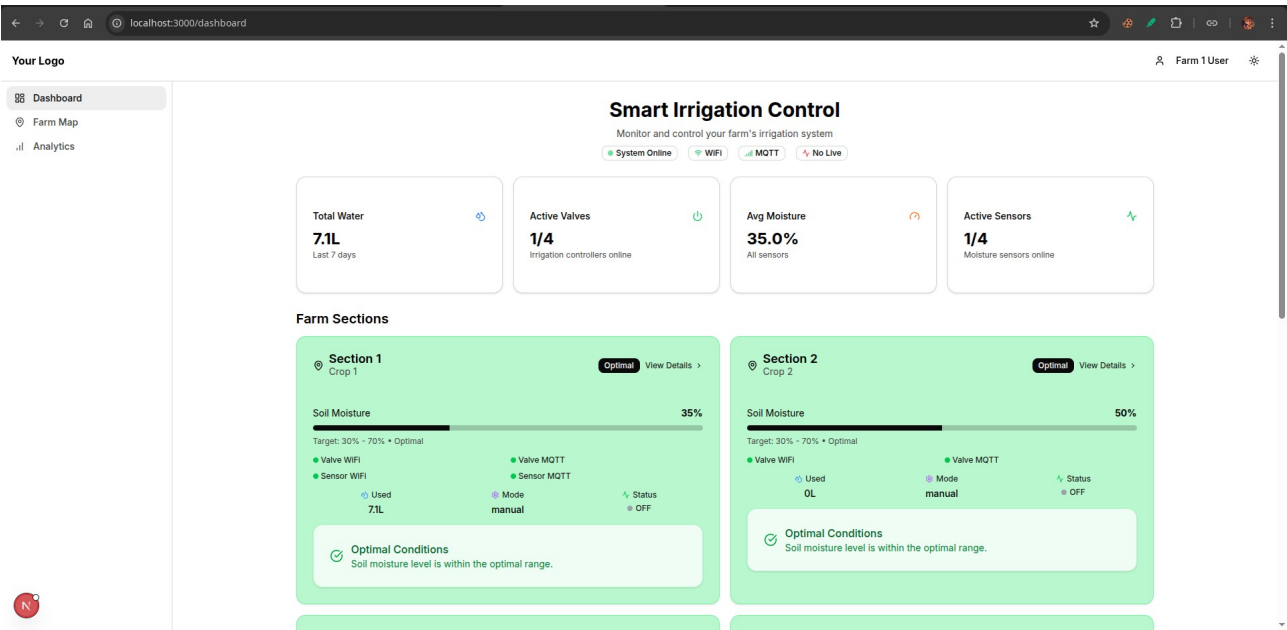


Figure 6.1 Dashboard of the Smart Irrigation Application

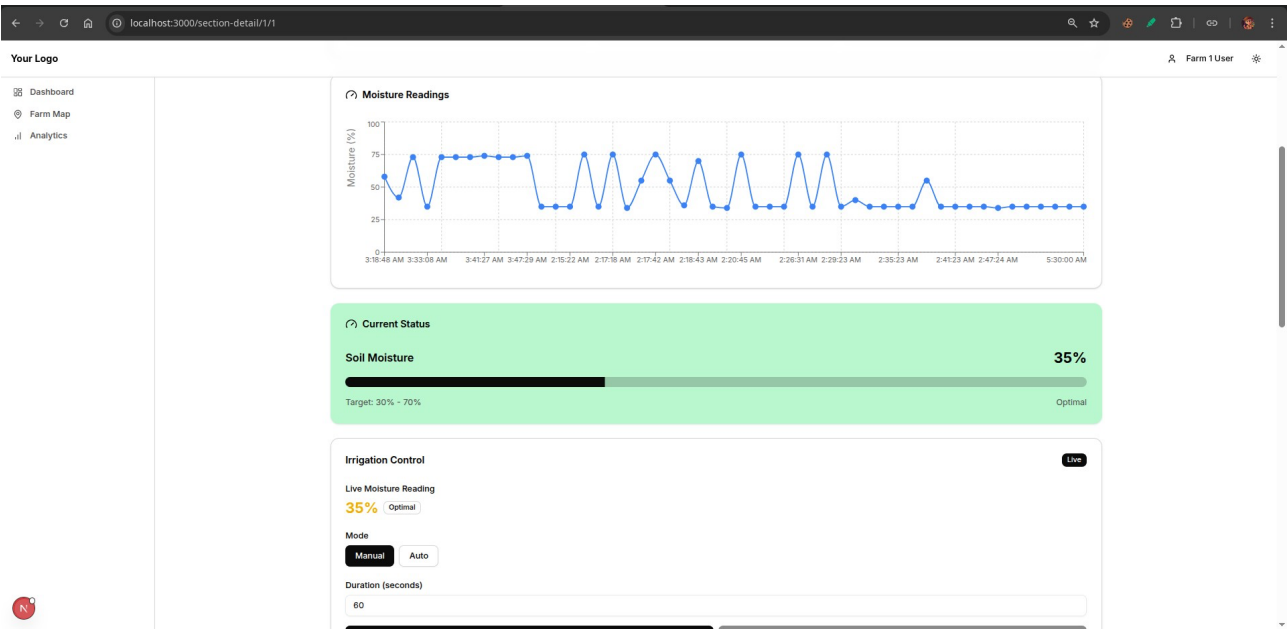


Figure 6.2 Detailed Section of the Smart Irrigation Application

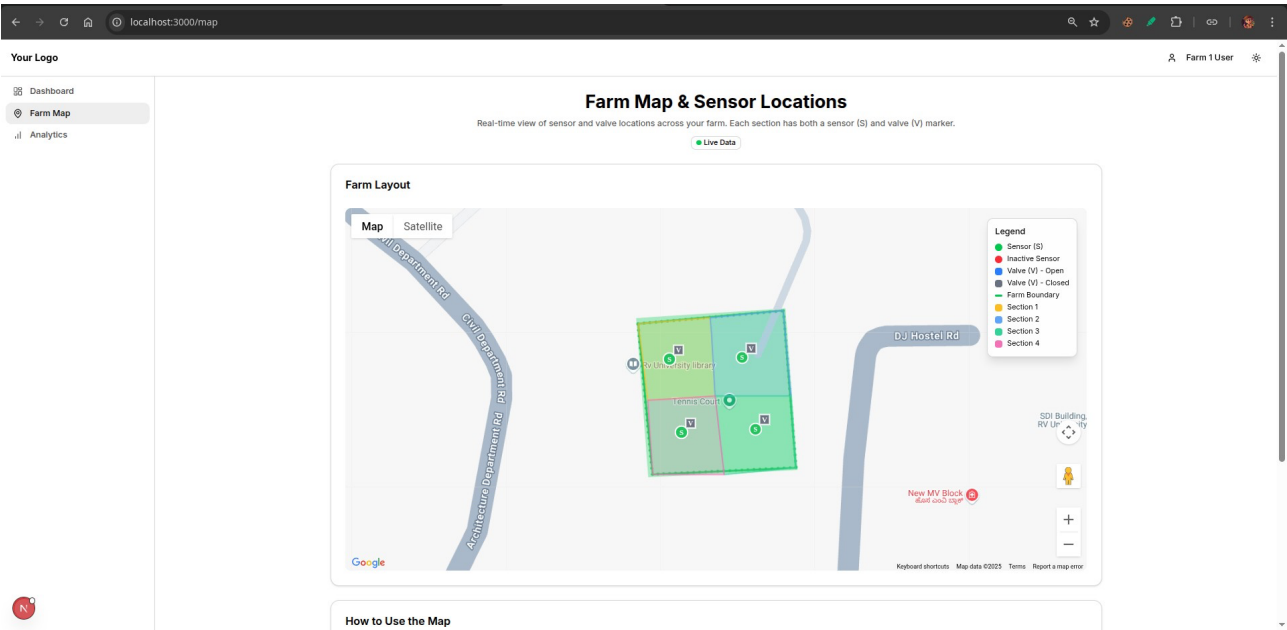


Figure 6.3 Farm Map For Smart Irrigation Application

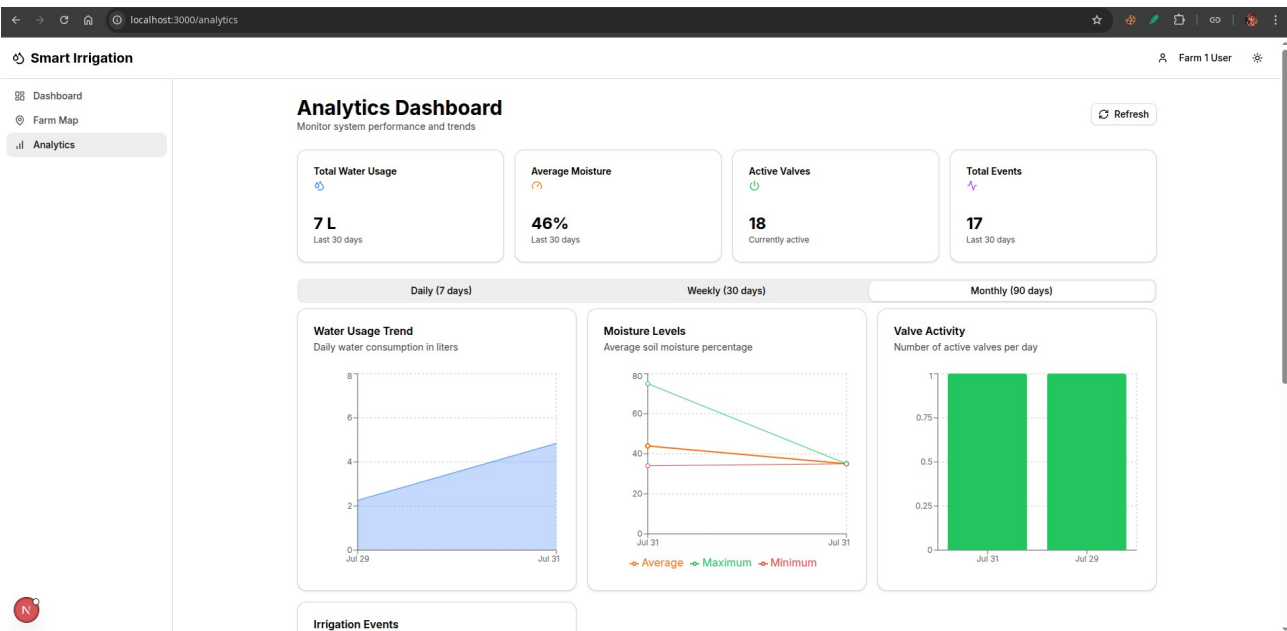


Figure 6.4 Analytics Page For Smart Irrigation

Chapter 7

Software Testing

7.1 Test Cases

- **Unit Testing (Firmware Level – ESP32):**

Each firmware function, such as moisture reading conversion, pulse counting for water flow, and MQTT publishing, was tested independently using serial logs. Known analog input values and pulse counts were simulated to ensure the accuracy of calculated outputs.

- **Integration Testing (Sensor + ESP32 + MQTT):**

Sensors were connected to the ESP32, and their real-time data was validated by monitoring MQTT topics. For example, after inserting the soil moisture sensor into dry and wet soil, the corresponding percentage was observed on the broker to confirm the full data path from sensing to publishing.

- **Backend Validation Testing (Node.js + PostgreSQL):**

API endpoints and database models were tested using Postman and terminal logs. Payloads were sent to ensure that data insertion (like moisture readings and irrigation logs) matched the schema and triggered appropriate database events.

Table 7.1 Test Case On Soil Moisture Monitoring

Test Case ID	Description	Expected Output	Actual Output	Result
TC - 01	Read moisture at dry condition	< 20% moisture	18%	Pass
TC - 02	Read moisture at fully wet soil	> 70% moisture	74%	Pass
TC - 03	Invalid sensor connection	Error log / zero value	0% with error flag	Pass
TC - 04	MQTT message format validation	JSON with section_device_id, value	Matches format	Pass

Table 7.2 Test Case on Irrigation Control (Auto/Manual)

Test Case ID	Mode	Moisture Value	Expected Valve State	Result
TC - 05	AUTO	25% (threshold 40%)	ON	Pass
TC - 06	AUTO	45% (threshold 40%)	OFF	Pass
TC - 07	MANUAL	Button “ON” clicked	ON	Pass
TC - 08	MANUAL	Button “OFF” clicked	OFF	Pass
TC - 09	AUTO	MQTT delay or loss	Retry or Timeout log	Pass

Table 7.3 Test Case on Water Flow Monitoring

Test Case ID	Input Flow Rate (L/min)	Expected Volume (L)	Measured Volume	Error %	Result
TC - 10	2 L/min for 5 mins	10.0	9.7	3%	Pass
TC - 11	1 L/min for 3 mins	3.0	2.9	3.3%	Pass
TC - 12	Flow = 0 (no movement)	0.0	0.0	0%	Pass
TC - 13	Irregular pulses	Consistent reading	Spike detected	-	Fail

Table 7.4 User Login for Smart Irrigation

Test Case ID	Description	Input	Expected Output	Result
TC - 14	Valid login credentials	Email + correct password	Redirect to dashboard	Pass
TC - 15	Invalid password	Email + wrong password	Show “Invalid credentials” message	Pass
TC - 16	Empty email or password field	Blank input	Show “Required fields” warning	Pass
TC - 17	Access dashboard without login	No token/session	Redirect to login page	Pass

7.2 Testing and Validations

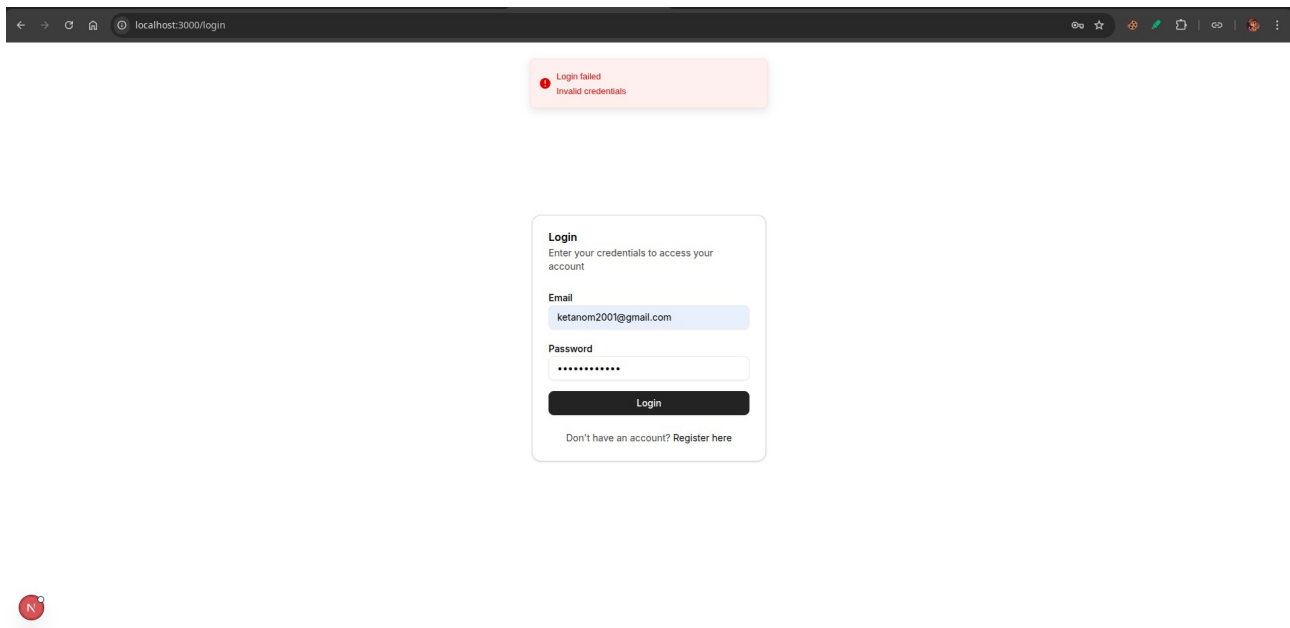


Figure 7.1 Validation of User Credentials in Login page

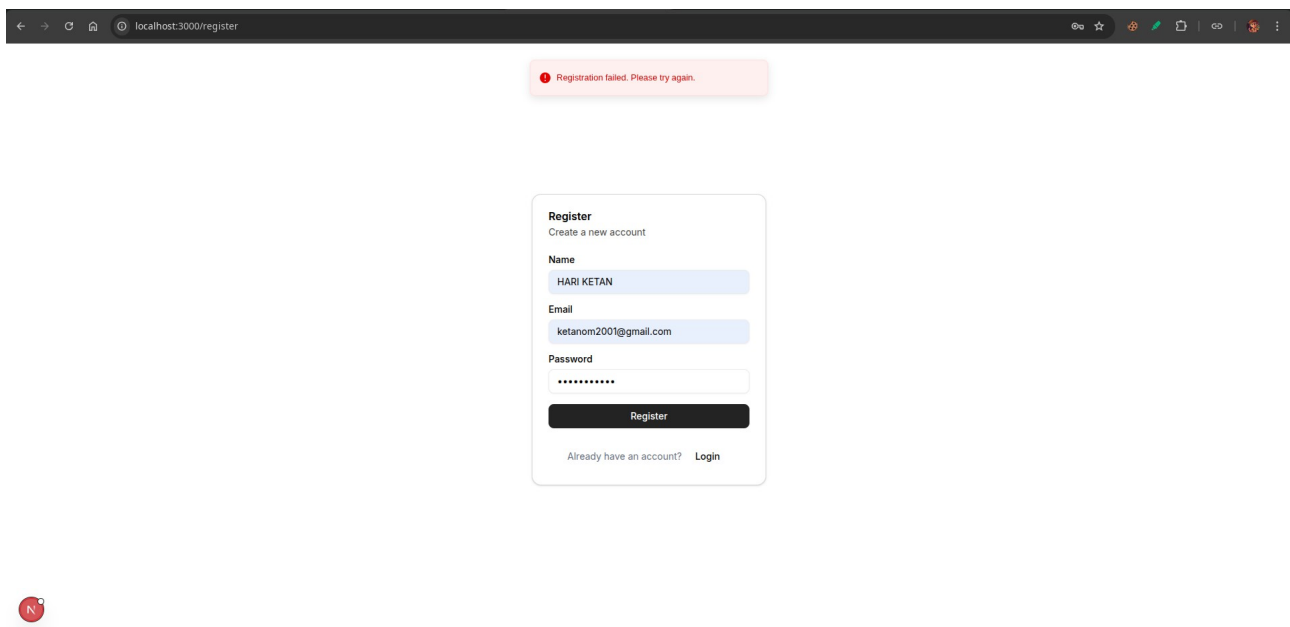


Figure 7.2 Validations of User Registration in Register page

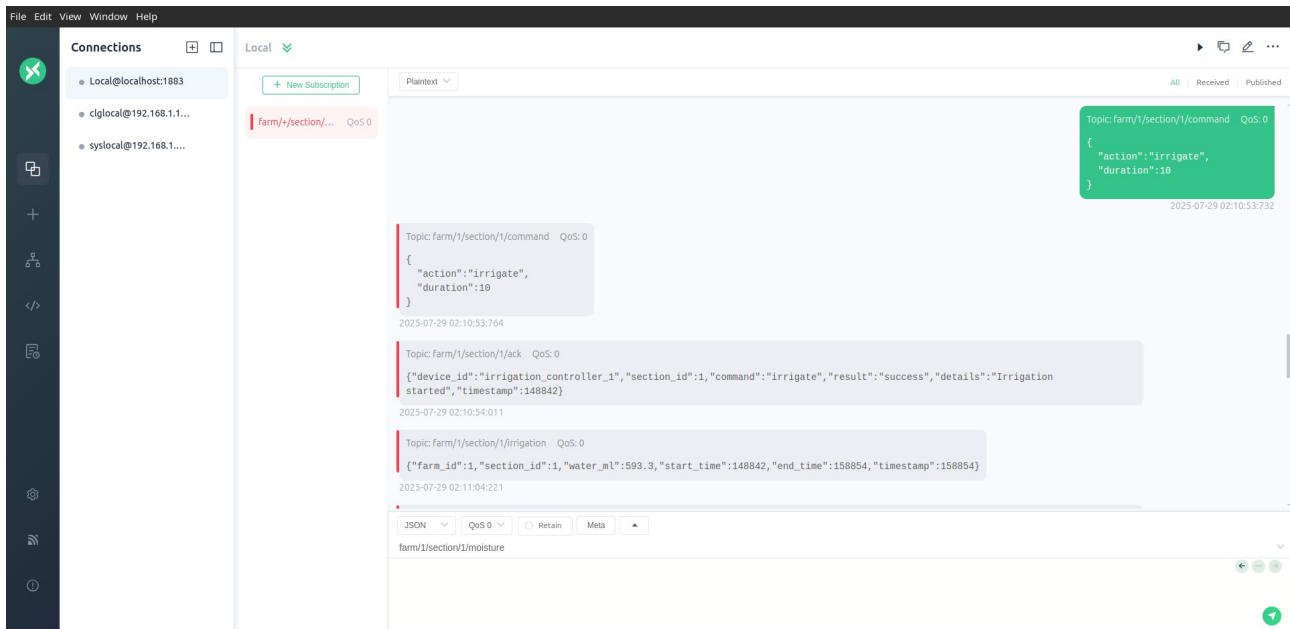


Figure 7.3 MQTTX Messages for Validating Irrigation Command

Chapter 8

Conclusion

The development of the smart irrigation system for arecanut farms has been a rewarding journey that combined technological innovation with real-world agricultural needs. The project was initiated with a clear goal to address the challenges faced by arecanut farmers in managing water usage efficiently and consistently. Through a structured approach, we were able to transform an idea into a fully functioning system that not only automates irrigation but also empowers farmers with live data and control.

Understanding the core problems such as over-irrigation, under-irrigation, and the lack of real-time soil moisture visibility. After a thorough literature review and technology analysis, we finalized a system design that integrates capacitive soil moisture sensors, a YF-S201 water flow sensor, ESP32 for data collection, and Raspberry Pi as the central controller. Communication between these components was established using the MQTT protocol, ensuring lightweight and fast data transfer.

Throughout the development, we tested each component in isolation and as part of the integrated system. From configuring sensors and setting up a secure database with PostgreSQL, to deploying the solution in a controlled farm environment, every step involved practical learning and debugging. The modular database schema was carefully designed to handle irrigation events, device acknowledgments, moisture readings, and section-specific control with traceable logs.

In conclusion, this project bridges the gap between technology and traditional agriculture. It promotes sustainable water usage, saves time and effort for farmers, and opens up new possibilities for automation in small to medium-sized farms. With its scalable design, the system can easily be extended to support more crops, integrate with weather APIs, or adopt AI-based irrigation forecasting in future versions. This marks just the beginning of how IoT can transform arecanut farming and beyond.

Chapter 9

Future Enhancements

- **Integration with Weather Forecasting APIs**

Incorporating real-time weather data can help the system make smarter decisions by avoiding irrigation during expected rainfall, conserving water, and improving timing accuracy.

- **Machine Learning for Predictive Irrigation**

Applying machine learning techniques can allow the system to learn from historical soil and irrigation data, enabling predictive scheduling and anomaly detection for better crop management.

- **Development of a Mobile Application**

A dedicated mobile app would allow farmers to monitor soil conditions, receive notifications, and control irrigation remotely, improving accessibility and user experience.

- **Support for Multi-Zone Irrigation**

Expanding the system to support multiple zones will enable efficient irrigation of larger farms with diverse crop types and soil conditions, offering greater flexibility.

- **Solar Power Integration**

Using solar panels as a power source can enhance the sustainability and reliability of the system, especially in rural or off-grid areas.

- **Automated Fault Detection and Alerts**

Adding logic to detect sensor or valve malfunctions and notify the user immediately can reduce downtime and prevent over/under-irrigation.

- **Integration with Fertigation Systems**

Future versions can also integrate fertigation units to automate the delivery of nutrients along with water, improving crop yield and reducing labor.

- **Scalability with Cloud-Based Infrastructure**

Migrating data storage and analytics to cloud platforms can improve performance, enable remote system updates, and support big-data analysis for multiple farms

Bibliography

- [1] K. R. SharathKumar, K. Mohan, and Nirisha, "Arecanut Crop Disease Prediction using Internet of Things and Machine Learning," *Journal of Science and Technology*, volume 5, number 3, pages 160–165, May 2020.
- [2] A. B. M. S. Ali, T. Hussain, N. Tantashutikun, N. Hussain, and G. Cocetta, "Application of Smart Techniques, Internet of Things and Data Mining for Resource Use Efficient and Sustainable Crop Production," *Agriculture*, volume 13, number 2, page 397, February 2023.
- [3] D. P. B. A. Dasanayaka, M. T. N. Kumara, and G. D. K. C. Morapitiya, "A Review of Smart Irrigation: The Future of Agriculture," *International Journal of Advanced Research in Engineering and Technology*, volume 12, number 1, pages 45–52, January 2021.
- [4] M. A. Othman and G. A. A. Al-Mekhlafi, "Impact of Over-Irrigation on Soil Properties and Nutrient Leaching: A Review," *Journal of Soil Science and Environmental Management*, volume 12, number 2, pages 58–67, 2021.
- [5] K. A. Patil and N. R. Kale, "A Model for Smart Farming using Internet of Things and Artificial Intelligence," *International Journal of Computer Applications*, volume 176, number 1, pages 24–29, 2020.
- [6] S. B. Goyal and A. Singh, "The Role of Internet of Things in Modernizing Traditional Agriculture Practices," *Journal of Agricultural Engineering and Technology*, volume 28, number 3, pages 210–219, 2023.
- [7] R. Jyothi, S. Rachana, R. Ranjitha, and R. Krishna Prasad, "Smart Irrigation System Based on Internet of Things," *International Journal of Advanced Science and Technology*, volume 29, number 3, pages 5730–5737, March 2020.
- [8] M. V. Gurao and U. B. Vaidya, "Internet of Things based Smart Irrigation System," *International Journal of Engineering Research and Technology*, volume 11, number 11, November 2022.
- [9] A. J. Rao, C. Bekal, Y. R. Manoj, R. Rakshitha, and N. Poornima, "Automatic Detection of Crop Diseases and Smart Irrigation Using Internet of Things and Image Processing," in *Advances in Communication and Computational Technology*, Springer, pages 343–353, 2021.
- [10] S. Ramya, A. M. Swetha, and M. Doraipandian, "Internet of Things Framework for Smart Irrigation Using Machine Learning Technique," *Journal of Computer Science*, volume 16, number 3, pages 355–363, March 2020.
- [11] M. T. Abdullah, M. Z. A. Aziz, and M. F. A. Aziz, "Internet of Things-Based Smart Farming System for Monitoring and Automation of Irrigation System," in *Proceedings of the 2022 Institute*

of Electrical and Electronics Engineers 12th Symposium on Computer Applications and Industrial Electronics, Penang, Malaysia, 2022, pages 119–124.

[12] H. Zheng and Y. Cheng, “Intelligent Water Resources Management Platform for Precision Irrigation Agriculture Based on Internet of Things,” *Neural Computing and Applications*, volume 34, pages 1–14, 2022.

[13] D. Thakur, Y. Kumar, and S. Vijendra, “Smart Irrigation and Intrusions Detection in Agricultural Fields Using Internet of Things,” *Procedia Computer Science*, volume 167, pages 154–162, 2020.

[14] M. N. H. Talukder, M. M. Islam, M. S. Uddin, M. A. Rahman, and M. S. Islam, “A Review on MQ Telemetry Transport Protocol for Internet of Things and Smart Applications,” *International Journal of Electrical and Computer Engineering*, volume 11, number 6, page 5313, 2021.

[15] A. Dahane, M. Benhaddou, and A. El Kettani, “Edge-Internet of Things-Cloud Architecture for Smart Irrigation Systems Based on Deep Learning,” *Agriculture*, volume 12, number 8, page 1198, August 2022.

[16] P. S. K. Reddy and P. V. S. S. R. Chandra Mouli, “A Comparative Study of Databases for Internet of Things Applications: Structured Query Language versus Not Only Structured Query Language,” in *Proceedings of the 2022 International Conference on Applied Artificial Intelligence and Computing*, 2022, pages 912–917.

[17] R. C. Puviarasi and V. S. Shankar, “Internet of Things-Based Smart Irrigation and Crop Monitoring System Using Machine Learning and Image Processing,” in *Proceedings of the 2022 International Conference on Computer, Communication and Signal Processing*, Chennai, India, 2022, pages 1–6.

[18] S. Kumar and V. K. Singh, “Soil Moisture-Based Automated Irrigation System: A Review,” *Journal of Critical Reviews*, volume 7, number 14, pages 1040–1045, 2020.

[19] Raja Muthuramalingam, Reshnuvi Rathnam Velu, Harshini Baskar, and Merun Hrithik Vellan Saminathan, “An Internet of Things-Based Smart Irrigation System,” *Engineering Proceedings*, volume 66, number 1, page 13, July 2024.

[20] M. S. Farooq, S. Riaz, A. Abid, K. Abid, and M. A. Naeem, “A Survey on the Role of Internet of Things in Agriculture for the Implementation of Smart Farming,” *Institute of Electrical and Electronics Engineers Access*, volume 7, pages 156237–156271, 2019.

- [21] L. García, L. Parra, J. M. Jimenez, J. Lloret, and P. Lorenz, "Internet of Things-Based Smart Irrigation Systems: An Overview on the Recent Trends on Sensors and Internet of Things Systems for Irrigation in Precision Agriculture," *Sensors*, volume 20, number 4, page 1042, February 2020.
- [22] A. Ali and G. Kalda, "Artificial Intelligence and Internet of Things for Sustainable Farming and Smart Agriculture," *Institute of Electrical and Electronics Engineers Access*, volume 11, pages 24866–24883, 2023.
- [23] Z. Kaleem, W. Khalid, M. A. Azam, and M. A. Khan, "An Internet of Things-based Automated Irrigation System for Efficient Water Management," *Sustainable Computing: Informatics and Systems*, volume 35, page 100732, September 2022.
- [24] A. Sharma, A. Jain, P. Gupta, and V. Chowdary, "A Review of Internet of Things based Applications in Agriculture Sector," *Journal of Ambient Intelligence and Humanized Computing*, volume 12, pages 1–20, 2021.
- [25] N. A. H. A. Alduais, J. A. M. Al-awadhi, M. A. Al-duais, A. H. M. Al-awadhi, and A. M. Al-ghaili, "Design and Implementation of a Smart Irrigation System Based on the Internet of Things and Cloud Computing," *Applied Water Science*, volume 13, number 1, page 15, January 2023.
- [26] A. Dahane, M. Benhaddou, and A. El Kettani, "Edge-Internet of Things-Cloud Architecture for Smart Irrigation Systems Based on Deep Learning," *Agriculture*, volume 12, number 8, page 1198, August 2022.
- [27] P. S. K. Reddy and P. V. S. S. R. Chandra Mouli, "A Comparative Study of Databases for Internet of Things Applications: Structured Query Language versus No Structured Query Language," in *Proceedings of the 2022 International Conference on Applied Artificial Intelligence and Computing*, 2022, pages 912–917.
- [28] R. C. Puviarasi and V. S. Shankar, "Internet of Things-Based Smart Irrigation and Crop Monitoring System Using Machine Learning and Image Processing," in *Proceedings of the 2022 International Conference on Computer, Communication and Signal Processing*, Chennai, India, 2022, pages 1–6.
- [29] S. Kumar and V. K. Singh, "Soil Moisture-Based Automated Irrigation System: A Review," *Journal of Critical Reviews*, volume 7, number 14, pages 1040–1045, 2020.
- [30] Raja Muthuramalingam, Reshnuvi Rathnam Velu, Harshini Baskar, and Merun Hrithik Vellan Saminathan, "An Internet of Things-Based Smart Irrigation System," *Engineering Proceedings*, volume 66, number 1, page 13, July 2024.
- [31] M. S. Farooq, S. Riaz, A. Abid, K. Abid, and M. A. Naeem, "A Survey on the Role of Internet of Things in Agriculture for the Implementation of Smart Farming," *IEEE Access*, volume 7, pages 156237–156271, 2019.

- [32] L. García, L. Parra, J. M. Jimenez, J. Lloret, and P. Lorenz, "Internet of Things-Based Smart Irrigation Systems: An Overview on the Recent Trends on Sensors and Internet of Things Systems for Irrigation in Precision Agriculture," *Sensors*, volume 20, number 4, page 1042, February 2020.
- [33] A. Ali and G. Kalda, "Artificial Intelligence and Internet of Things for Sustainable Farming and Smart Agriculture," *IEEE Access*, volume 11, pages 24866–24883, 2023.
- [34] Z. Kaleem, W. Khalid, M. A. Azam, and M. A. Khan, "An Internet of Things-Based Automated Irrigation System for Efficient Water Management," *Sustainable Computing: Informatics and Systems*, volume 35, page 100732, September 2022.
- [35] A. Sharma, A. Jain, P. Gupta, and V. Chowdary, "A Review of Internet of Things-Based Applications in Agriculture Sector," *Journal of Ambient Intelligence and Humanized Computing*, volume 12, pages 1–20, 2021.
- [36] N. A. H. A. Alduais, J. A. M. Al-awadhi, M. A. Al-duais, A. H. M. Al-awadhi, and A. M. Al-ghaili, "Design and Implementation of a Smart Irrigation System Based on the Internet of Things and Cloud Computing," *Applied Water Science*, volume 13, number 1, page 15, January 2023.
- [37] I. Ravi, G. Prabhu, P. S. Shelake, R. Selvarajan, and V. B. Patel, "Internet of Things Enabled Soil Moisture Sensor Based Irrigation in Banana," *Indian Horticulture*, volume 69, number 6, pages 17–21, January 2025.
- [38] E. Chandu and M. K. Shukla, "Internet of Things-Based Smart Irrigation System for Mango Orchard," in *Artificial Intelligence and Machine Learning in Satellite Data Processing and Services*, Springer, pages 61–69, January 2023.
- [39] M. A. Al-Ghobari, M. S. A. El-Marazky, and A. M. El-Enany, "An Internet of Things-Based Smart Irrigation System for Improving Water Use Efficiency in Date Palm Groves," *Journal of King Saud University – Engineering Sciences*, volume 34, number 6, pages 367–375, 2022.
- [40] K. S. S. Iyer, "A Study on the Technological Gaps in Arecanut Cultivation in South India," *Indian Journal of Agricultural Research*, volume 56, number 2, pages 150–155, 2022.
- [41] I. Sommerville, *Software Engineering*, 10th edition. Pearson, 2016.
- [42] A. Rajak, V. Kumar, and S. Singh, "Internet of Things-Enabled Smart Irrigation System for Precision Agriculture," in *Smart Systems and Internet of Things: Innovations in Computing*, Singapore: Springer, 2023, pages 143–154.
- [43] M. A. R. Khan and S. M. R. Islam, "On-Premises versus Cloud-Based Internet of Things Platforms for Smart Farming: A Performance Analysis," *Computers and Electronics in Agriculture*, volume 187, page 106257, 2021.
- [44] G. Li, S. He, and Y. Zhang, "A Volumetric Water-Saving Irrigation Control System Based on Internet of Things," *Water Resources Management*, volume 35, pages 501–514, 2021.

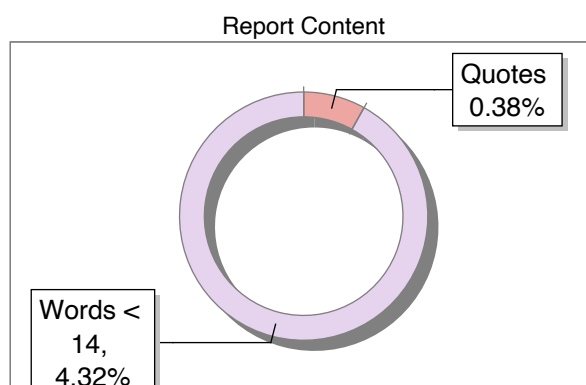
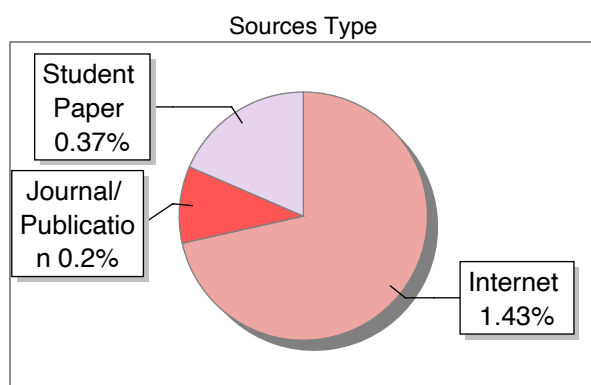
- [45] J. M. S. P. Johnson, M. A. J. Christy, and M. P. G. Raj, "A Microservices-Based Architecture for Smart Agriculture Internet of Things Platforms," *IEEE Internet of Things Journal*, volume 9, number 12, pages 9845–9854, 2022.
- [46] S. B. Pawar and P. Rajput, "Cloud and Internet of Things-Based Smart Irrigation System for Sustainable Agriculture," in *Proceedings of the 2021 International Conference on Communication Information and Computing Technology*, Mumbai, India, 2021, pages 1–6.
- [47] T. H. T. Nguyen, D. T. Dao, and V. D. Hoang, "Performance Evaluation of ESP32 and ESP8266 for Internet of Things-Based Sensor Networks," in *Proceedings of the 2020 International Conference on Advanced Technologies for Communications*, 2020, pages 182–186.
- [48] V. C. M. L. Singh, "Using Raspberry Pi as a Low-Cost Gateway for Agricultural Internet of Things Applications," *Journal of Sensor Technology and Applications*, volume 10, number 1, pages 12–20, 2024.
- [49] C. A. A. C. Portocarrero, J. D. V. C. Paucar, and J. M. L. L. Requena, "Development of a Real-Time Monitoring Web Application for Agriculture Using React and Node.js," *Journal of Physics: Conference Series*, volume 1845, number 1, page 012028, 2021.
- [50] K. G. S. G. S. R. R. B. Bandara, "Building Scalable Internet of Things Backends with Node.js and Message Queuing Telemetry Transport," *International Journal of Advanced Computer Science and Applications*, volume 13, number 5, 2022.
- [51] J. Davis, "Time-Series Data Storage for Internet of Things: A Comparison of InfluxDB and PostgreSQL," *Journal of Cloud Computing: Advances, Systems and Applications*, volume 10, number 1, pages 1–17, 2021.
- [52] A. M. H. Hassan, "Python for Internet of Things: A Practical Guide for Data-Centric Application Development," *IEEE Potentials*, volume 42, number 2, pages 29–35, 2023.
- [53] M. S. Alam, "A Comparative Analysis of Capacitive and Resistive Soil Moisture Sensors for Internet of Things Applications," *Sensors and Actuators A: Physical*, volume 331, page 112999, 2021.
- [54] "PlatformIO: An Open Source Ecosystem for Internet of Things Development," [Online]. Available: <https://platformio.org/>. [Accessed: July 30, 2025].
- [55] IEEE Standard for System and Software Requirements Specifications, IEEE Standard 830-1998, 1998.
- [56] G. K. Wat, "A Framework for Eliciting Non-Functional Requirements in Agile Internet of Things Projects," in *Proceedings of the 2023 IEEE 5th International Conference on Information and Computer Technologies*, 2023, pages 220–225.
- [57] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th edition. McGraw-Hill, 2020.

- [58] T. K. Lim, "Defining Functional Modules for a Generic Smart Farming Platform," *International Journal on Smart Sensing and Intelligent Systems*, volume 15, number 1, 2022.
- [59] J. Nielsen, *Usability Engineering*. Morgan Kaufmann, 1993.
- [60] V. K. Kumar, "Economic Constraints and Technology Adoption in Smallholder Farming," *Journal of Development Economics*, volume 152, page 102690, 2021.
- [61] P. J. K. Patel, "Risk Management in Internet of Things Projects: A Focus on Assumptions and Dependencies," *PM World Journal*, volume 11, number 8, August 2022.
- [62] M. A. Ali, "Design Principles for User Interfaces in Agricultural Decision Support Systems," *Computers and Electronics in Agriculture*, volume 198, page 107075, 2022.
- [63] A. D. Dutta, "Hardware-Software Interface Design for Embedded Internet of Things Systems," *ACM Transactions on Embedded Computing Systems*, volume 22, number 3, pages 1–25, 2023.
- [64] S. Ray, "A Survey of Weather Application Programming Interfaces for Integration in Smart Agriculture Systems," *Agri-Informatics Journal*, volume 2, number 1, pages 34–45, 2024.
- [65] Y. J. Yang, "Performance and Latency Analysis of Edge-Based Internet of Things Control Systems," *IEEE Transactions on Industrial Informatics*, volume 18, number 5, pages 3478–3487, 2022.
- [66] S. Misra, "Scalability Challenges in Agricultural Internet of Things Sensor Networks," *IEEE Communications Magazine*, volume 60, number 7, pages 78–83, 2022.
- [67] A. G. Gonzales, "Calibration and Accuracy Validation of Low-Cost Capacitive Soil Moisture Sensors," *Journal of Irrigation and Drainage Engineering*, volume 147, number 9, page 04021035, 2021.
- [68] A. P. Papageorgiou, "Reliability and Fault Tolerance in Mission-Critical Internet of Things Systems," *IEEE Systems Journal*, volume 16, number 1, pages 1256–1267, 2022.
- [69] K. L. Kumar, "Data Logging and Retention Policies for Internet of Things Data," *Journal of Data Management and Policy*, volume 4, number 1, pages 22–30, 2025.
- [70] D. A. Norman, *The Design of Everyday Things: Revised and Expanded Edition*. Basic Books, 2013.

Submission Information

Author Name	1RV23MCA039_HARI_KETAN
Title	1RV23MCA039_HARI_KETAN
Paper/Submission ID	4206776
Submitted by	ushaj@rvce.edu.in
Submission Date	2025-08-05 09:16:42
Total Pages, Total Words	51, 11427
Document type	Project Work

Result Information

Similarity **2 %**

Exclude Information

Quotes	Excluded
References/Bibliography	Excluded
Source: Excluded < 14 Words	Excluded
Excluded Source	0 %
Excluded Phrases	Not Excluded

Database Selection

Language	English
Student Papers	Yes
Journals & publishers	Yes
Internet or Web	Yes
Institution Repository	Yes

A Unique QR Code use to View/Download/Share Pdf File





DrillBit Similarity Report

2	7	A	A-Satisfactory (0-10%) B-Upgrade (11-40%) C-Poor (41-60%) D-Unacceptable (61-100%)
SIMILARITY %	MATCHED SOURCES	GRADE	

LOCATION	MATCHED DOMAIN	%	SOURCE TYPE
1	docplayer.info	<1	Internet Data
2	REPOSITORY - Submitted to Exam section VTU on 2024-07-31 16-47 900462	<1	Student Paper
3	qdoc.tips	<1	Internet Data
4	www.mdpi.com	<1	Publication
5	coek.info	<1	Internet Data
6	management.org	<1	Internet Data
7	govt.westlaw.com	<1	Internet Data